

LAB MANUAL

in

CompEng 412 EMBEDDED SYSTEM

ENGR. JOLAN BACCAY SY
Asst. Professor

LABORATORY EXERCISES

in

EMBEDDED SYSTEM

- CompEng 412-



COLLEGE OF COMPUTING STUDIES, INFORMATION AND COMMUNICATION TECHNOLOGY



ENGR.JOLAN B.SY , R.ECE,M.ECE

Name: _____

Course/Year/Block: _____

Group No.: _____

_____ Semester, School Year _____

TABLE OF CONTENTS

Laboratory Exercise No.	Title	Page
Experiment 1	Introduction to Arduino Board	
	Arduino IDE	
Experiment 2	LED applications	
	Part 1. BLINK LED	
	Part 2: Blinking 5 LEDs	
Experiment 3	Latch and Switch Button	
Experiment 4	Potentiometer Applications	
	Part 1: Potentiometer as Light dimmer	
	Part 2 – Potentiometer value used as delay	
Experiment 5	Light Dependent Resistor	
Experiment 6	Piezoelectric (Buzzer)	
Experiment 7	Temperature Sensor Applications	
Experiment 8	Relay Applications	
Experiment 9	Application and Interfacing of DC Motor	
Experiment 10	Application and Interfacing of Servo Motor	
Experiment 11	LCD Display	
Experiment 12	Security System using Keypad with LCD	
Experiment 13	Ultrasonic Sensor With Buzzer	
Experiment 14	Ultrasonic Sensor with LCD	
Experiment 15	DHT sensors with LCD	
Experiment 16	Soil Moisture Sensor with Pump	
Experiment 17	PIR Sensor with Buzzer	
Experiment 18	Automatic Hand Sanitizer	
Experiment 19	Single Station Automatic Hand Wash	
Experiment 20	Stick for the blind	
Experiment 21	Water Level Alerting system – SMS	
Experiment 22	Infrared Temperature sensor	
Experiment 23	Automated Disinfection Cabin	
Experiment 24	GSM Based Security System	
Experiment 25	COVID 19 Patient Tracking System	
Experiment 26	GSM based Automatic Plant Irrigation System	
Experiment 27	Alcohol Sensor Detector with SMS	
Experiment 28	Heart Rate Monitoring System with SMS and Display	
Experiment 29	Automated Digital Counter	
Experiment 30	Security System Part 1- RFID Door Lock with Display	
	Security System Part 2. IR Remote Home Automation	
	Rubrics	
	Summary of Scores	

Experiment 1

Introduction to Arduino

Learning Outcomes

1. Understand what is meant by Microcontroller, Microprocessor, Embedded System, Integrated Development Environment (IDE), and Arduino.
2. Understand the layout of Arduino UNO board.
3. Understand Arduino IDE, and how to write, compile, and upload a code to Arduino.
4. Understand switch bouncing problem and how to overcome it by simple coding solution.
5. Able to install Arduino IDE and Proteus Simulator .

Working Principle

Part 1. Introduction to Arduino Technology

Arduino UNO is an open source microcontroller board based on the ATmega328P. The board has 14 digital input/output pins that 6 can be used as PWM outputs, 6 analog inputs, a 16 MHz ceramic resonator (CSTCE16MOV53-R0), it can be powered by the USB cable or by external power source such as battery or dc charger. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started. The board is programmable with the Arduino IDE (Integrated Development Board)

Easy to use and the best board to get started with electronics and coding.

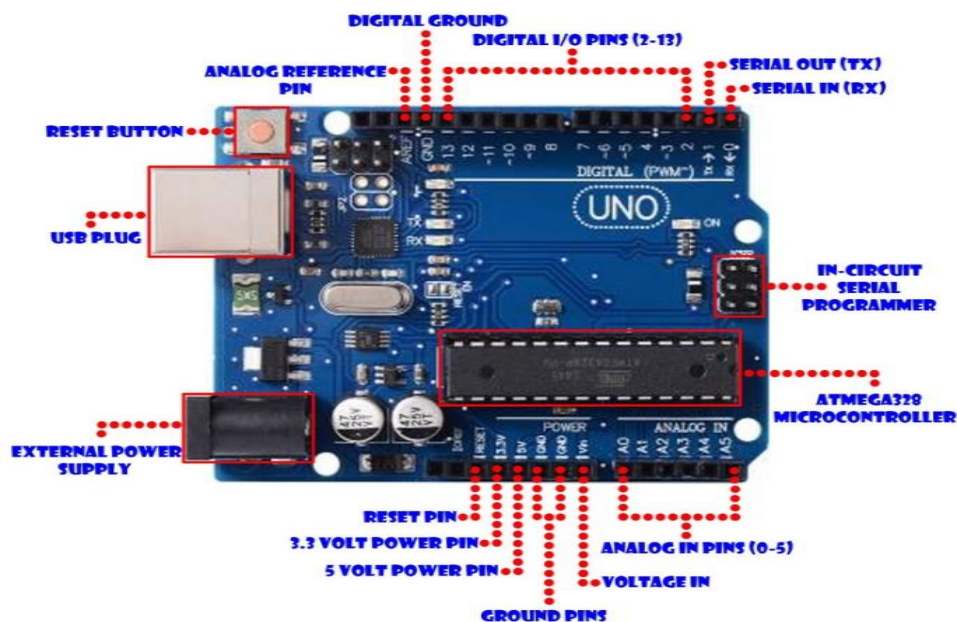


Figure 1: Arduino Board

Arduino IDE

The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. It runs on Windows, Mac OS X, and Linux. This software can be used with any Arduino board.

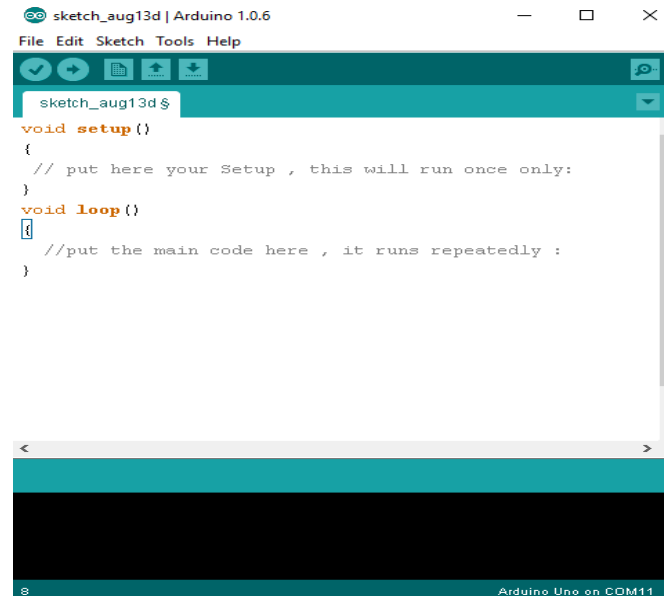


Figure 2: Arduino IDE

USB A Plug

USB cable A can be used to power Arduino Board from your computer. All you need to do is connect the USB cable to the USB connection.

External Power Supply

Arduino boards can also be powered directly from the DC Charger with less than 17 V and Greater than 6V power supply.

Reset Button

You can reset your Arduino board, i.e., start your program from the beginning. You can reset the UNO board in two ways.

Pins (3.3, 5, GND, Vin)

3.3V – Supply 3.3 output volt

5V – Supply 5 output volt

Most of the components used with Arduino board works fine with 3.3 volt and 5 volt.

GND (Ground) – There are several GND pins on the Arduino, any of which can be used to ground your circuit.

Vin – This pin also can be used to power the Arduino board from an external power source, like AC mains power supply.

Analog pins

The Arduino UNO board has six analog input pins A0 through A5. These pins can read the signal from an analog sensor like the humidity sensor or temperature sensor and convert it into a digital value that can be read by the microprocessor.

Digital I/O

The Arduino UNO board has 14 digital I/O pins. These pins can be configured to work as input digital pins to read logic values (0 or 1) or as digital output pins.

AREF

AREF stands for Analog Reference. It is sometimes, used to set an external reference voltage (between 0 and 5 Volts) as the upper limit for the analog input pins.

Circuit serial programming (ICSP)

Microcontroller to be programmed without disconnecting from the circuitry.

TX and RX LEDs

On your board, you will find two labels: TX (transmit) and RX (receive). They appear in two places on the Arduino UNO board. First, at the digital pins 0 and 1, to indicate the pins responsible for serial communication. Second, the TX and RX led. The TX led flashes with different speed while sending the serial data. The speed of flashing depends on the baud rate used by the board. RX flashes during the receiving process.

Main microcontroller

Each Arduino board has its own microcontroller. You can assume it as the brain of your board. The main IC (integrated circuit) on the Arduino is slightly different from board to board. The microcontrollers are usually of the ATMELE Company. You must know what IC your board has before loading up a new program from the Arduino IDE. This information is available on the top of the IC. For more details about the IC construction and functions, you can refer to the data sheet.

Part 2. Arduino Installation Guide

Steps to set up Arduino IDE on computer and to see if the board read in serial interface

Step 1 – First you must have your Arduino board and usually they will give you with the USB cable . There are two different types of cable depends on Arduino to use.



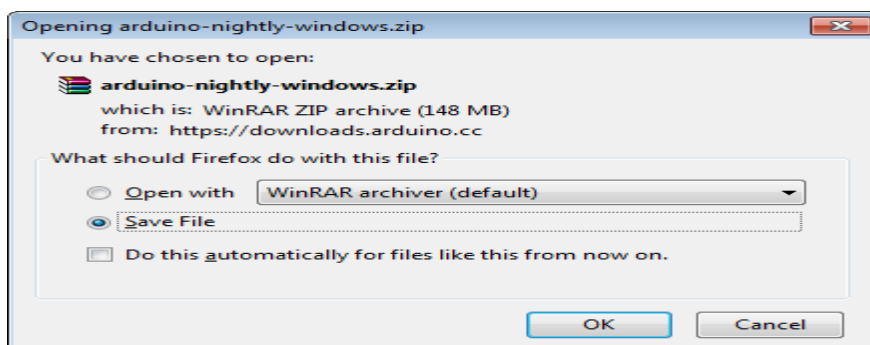
USB A- USB B



USB A-Mini B

Step 2 – Download Arduino IDE Software.

You can get different versions of Arduino IDE from the [Download page](#) on the Arduino Official website. You must select your software, which is compatible with your operating system (Windows, IOS, or Linux). After your file download is complete, unzip the file.

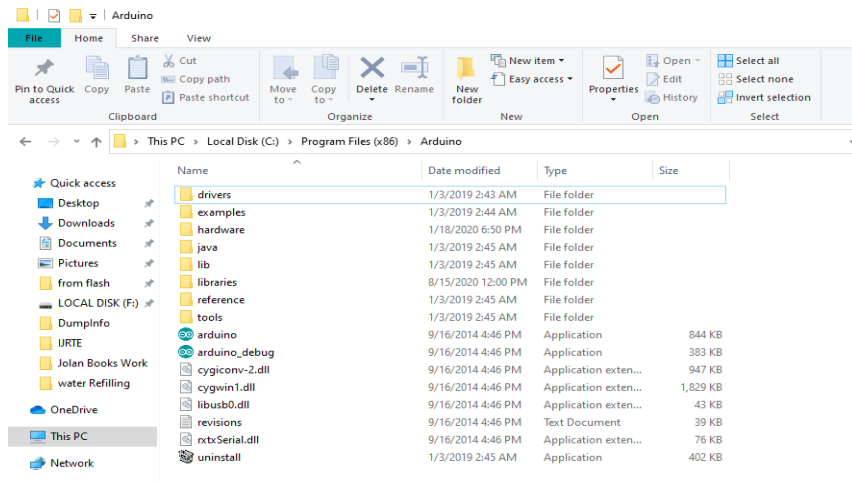


Step 3 – Power up your board.

Connect the Arduino board to your computer using the USB cable. The green power LED (labeled PWR) should glow.

Step 4 – Launch Arduino IDE.

After your Arduino IDE software is downloaded, you need to unzip the folder. Inside the folder, you can find the application icon with an infinity label (application.exe). Double-click the icon to start the IDE.



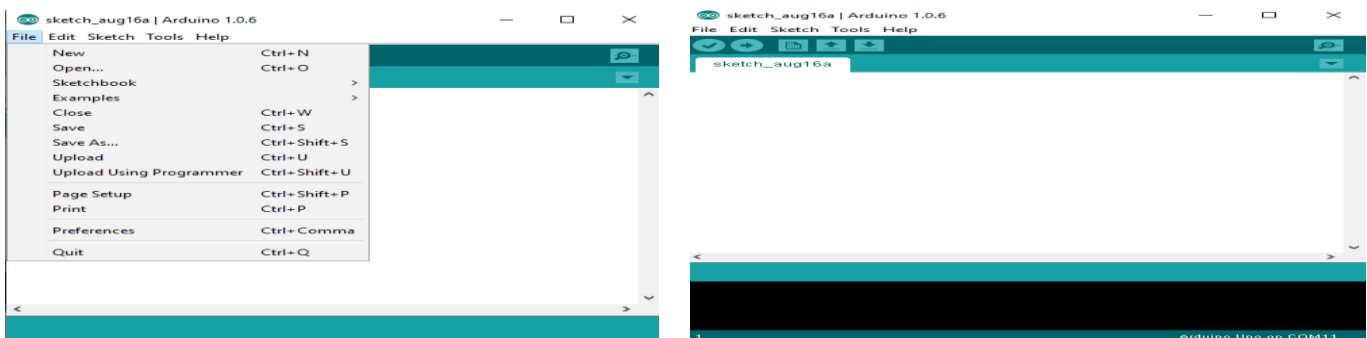
Step 5 – Open your first project.

Once the software starts, you have two options –

Create a new project.

Open an existing project example.

To create a new project, select File → **New**.



To open an existing project example, select File → Example → Basics → Blink.

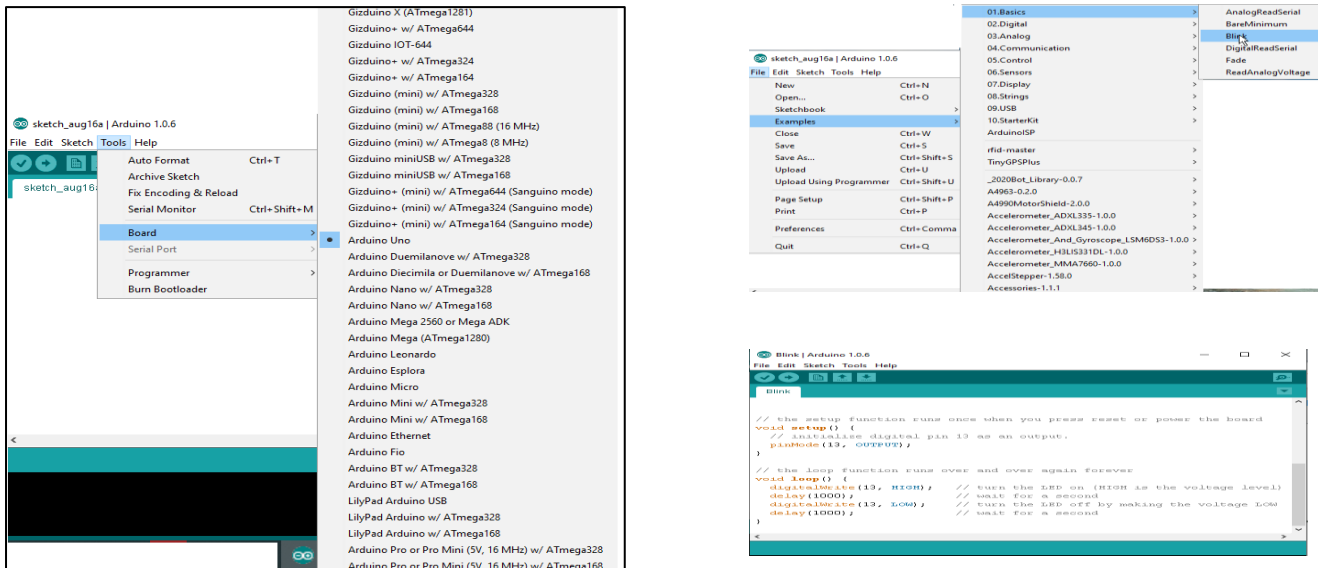
Here, we are selecting just one of the examples with the name **Blink**. It turns the LED on and off with some time delay. You can select any other example from the list.

Step 6 – Select your Arduino board.

To avoid any error while uploading your program to the board, you must select the correct Arduino board name, which matches with the board connected to your computer.

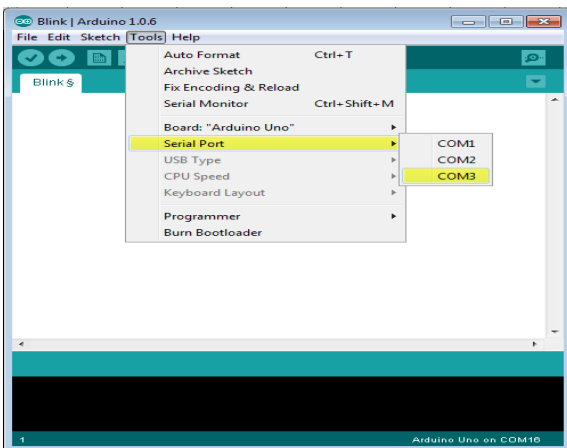
Go to Tools → Board and select your board.

Here, we have selected Arduino Uno board according to our tutorial, but you must select the name matching the board that you are using.



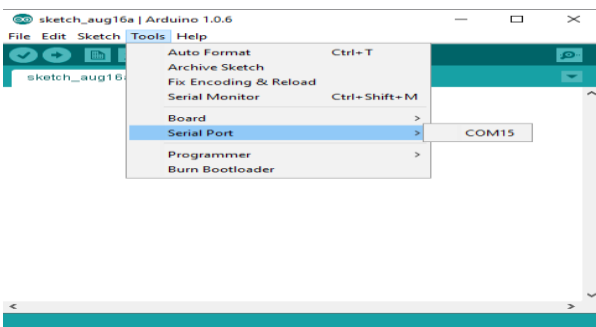
Step 7 – Select your serial port.

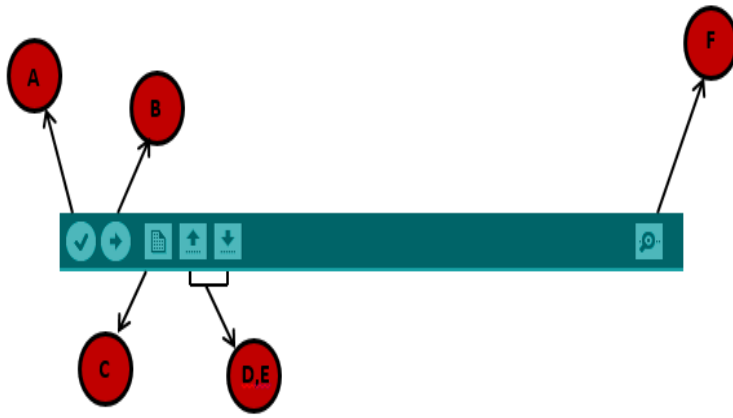
Select the serial device of the Arduino board. Go to **Tools → Serial Port** menu. This is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports). To find out, you can disconnect your Arduino board and re-open the menu, the entry that disappears should be of the Arduino board. Reconnect the board and select that serial port.



Step 8 – Upload the program to your board.

Before explaining how we can upload our program to the board, we must demonstrate the function of each symbol appearing in the Arduino IDE toolbar.





- A – Used to check if there is any compilation error.
- B – Used to upload a program to the Arduino board.
- C – Shortcut used to create a new sketch.
- D – Used to directly open one of the example sketch.
- E – Used to save your sketch.
- F – Serial monitor used to receive serial data from the board and send the serial data to the board.

Now, simply click the "Upload" button in the environment. Wait a few seconds; you will see the RX and TX LEDs on the board, flashing. If the upload is successful, the message "Done uploading" will appear in the status bar.

Discussion/Observation:

Conclusion :

Assessment task :

1. Enumerate and differentiate different type of Arduino.

[illegible]

List down different inputs and outputs that can be interface in Arduino

[illegible]

Helpful Tools

1. <https://library.io/>
Simulates your Arduino code
2. <https://fritzing.org/>
Creates virtual circuits (Schematic for your real circuits)
 - Has a PCB view to put components on a PCB with autoroute feature.
 - Share your design (if you want)
 - Contain a collection of projects for beginners and advanced users

Home Work

1. Review Arduino Language:

<http://arduino.cc/en/Reference/HomePage>

Read the PDF of reviewing the most important functions and instructions.

References :

Mastering Arduino basics in 30 Days by Engr. Jolan B. Sy

<http://arduino.cc>

<http://www.ladyada.net/learn/arduino/index.html>

<http://learn.adafruit.com>

<https://www.instructables.com/Intro-to-Arduino/>

Experiment 2

Blinking and trailing LED

Learning Outcomes:

1. Understand the basic principle Arduino and different components use.
2. Able to interface LED to Arduino
3. Deal with LED using Arduino digital output pins.
4. Understand the difference between Pull-up and Pull-down resistors configurations.
5. Write Arduino Code for LED blinking and Trailing LED's
6. Analyze and evaluate different problems and its applications.
7. learn how to implement digital output from the Arduino board.

Principle of the Activity

Introduction

LEDs are small, powerful lights that are used in many different applications. To start off, we will work on blinking an LED, the Hello World of microcontrollers. That's right - it's as simple as turning a light on and off. It might not seem like much, but establishing this important baseline will give you a solid foundation as we work toward more complex experiments.

Part 1: BLINKING LED

Parts Needed

List of Components

- 1 Arduino Uno
- 1 LED (green)
- 1 220 Ω
- 2 Connecting Wires (M/M)
- 1 Bread Board

LED Pinout

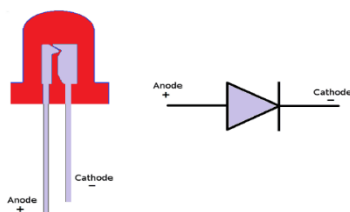


Figure 1.1: LED Pinout

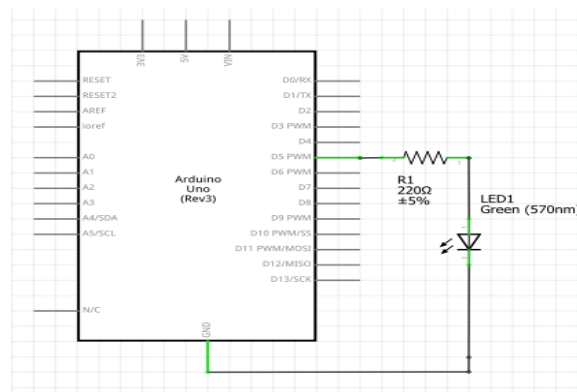


Figure 2.2 –Blinking LED Schematic Diagram

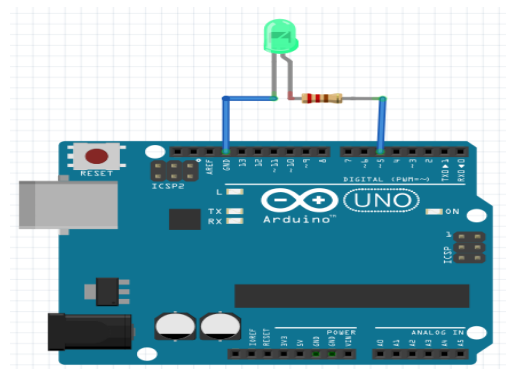


Figure 2.3 –Blinking LEDs Pictorial Circuit Diagram

Circuit Construction and Discussion

Connect the one end of the resistor to the ground pin of the arduino and the other end to the cathode of the LED .The anode pin of the LED will be connected to PIN 5 .

The CODE –Blink LED

Sketch

```
// Assign connection to Arduino Uno pins
const int LED = 5;
void setup() {
// Initialize digital pin 5 mode as output
pinMode(LED, OUTPUT);
}
void loop() {
digitalWrite(LED, HIGH); // Turn LED on
delay(1000); // Keep LED on for 1000 ms
digitalWrite(LED, LOW); // Turn LED off
delay(1000); // Keep LED off for 1000 ms
}
```

Part 2: Trailing LEDs

This Activities show trailing LEDs

List of Components

- 1 Arduino Uno
- 5 LED
- 5 220 Ω
- 12 Connecting Wires
- 1 Bread Board

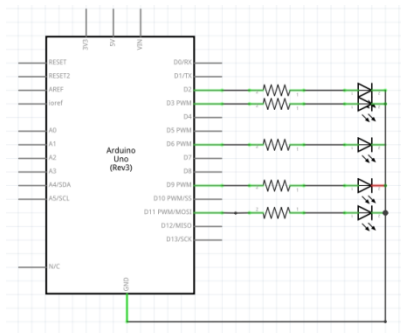


Figure 2.4 : Trailing LEDs Schematic Diagram

Circuit Construction

Connect one end of all resistor to the ground pin of the Arduino and the other end to the cathode of the LED . The anode pin of the LED will be connected to PIN 5,Pin 6, Pin 9, Pin 10 , Pin 11(Follow the circuit Diagram)

Sketch

```
// Assign connection to Arduino Uno pins
const int LED_1 =2;
const int LED_2 =3;
const int LED_3 =6;
const int LED_4 = 9;
const int LED_5 = 11;
void setup() {
// Set the digital pin mode as output
pinMode(LED_1, OUTPUT);
pinMode(LED_2, OUTPUT);
pinMode(LED_3, OUTPUT);
pinMode(LED_4, OUTPUT);
pinMode(LED_5, OUTPUT);
}
```

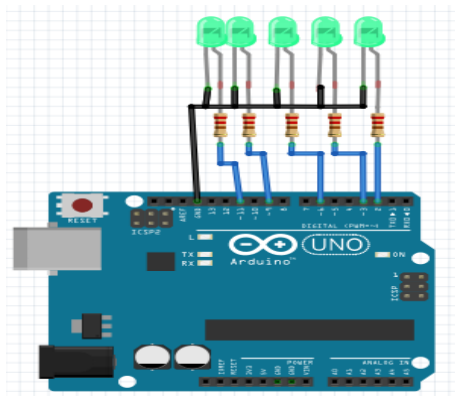


Figure 2.5–Trailing LED Pictorial Circuit Diagram

```

void loop() {
  // Turn the LEDs on
  digitalWrite (LED_1, HIGH);
  delay(1000);
  digitalWrite (LED_2, HIGH);
  delay(1000);
  digitalWrite (LED_3, HIGH);
  delay(1000);
  digitalWrite (LED_4, HIGH);
  delay(1000);
  digitalWrite (LED_5, HIGH);
  delay(1000); // Wait for a second
  // Turn the LEDs off
  digitalWrite(LED_1, LOW);
  delay(1000);
  digitalWrite(LED_2, LOW);
  delay(1000);
  digitalWrite(LED_3, LOW);
  delay(1000);
  digitalWrite(LED_4, LOW);
  delay(1000);
  digitalWrite(LED_5, LOW);
  delay(1000); // Wait for a second
}

```

Discussion/Observation:

Conclusion :

Assessment Task

Change LED status with push button press

References :

<http://arduino.cc>
<http://www.ladyada.net/learn/arduino/index.html>
<http://learn.adafruit.com>

Experiment 3

Latch and Switch Button

Learning Outcomes

1. Understanding Digital Read
2. Understanding IF-ELSE conditions
3. Control the color of the RGB LED
4. Use push buttons to control on RGB LED.
5. Learn about interfacing the button with Arduino using the Arduino digitalRead function

Principle of Activity :

Generally, switches are categorized into two types based on performance & holding current conditions like latching and non-latching. There are different kinds of switches under different categories like electronic, normally open, normally closed, momentary, etc. The working of latching and non-latching switches are; once a latching switch is activated, it stays on until the pressure is disabled or removed whereas a non-latching switch turns ON only when pressed and turns OFF when removed

Problem :

1. Switch 1 when pressed and released
Will keep the Red Led ON. When pressed
And release again will make the Red LED off
2. Switch 2 when pressed will switch on
The green LED . When released will switch off.

List of Components

- 1 Arduino Uno
- 2 Push Button Switch
- 2 220 ohms Resistors
- 1 Green Led
- 1 Red Led
- 1 Bread Board

Sketch

// Assign connection to Arduino Uno pins

```
const int LED_RED = 4;  
const int LED_GREEN = 5;  
const int SWITCH_1 = 2;  
  
const int SWITCH_2 = 3;
```

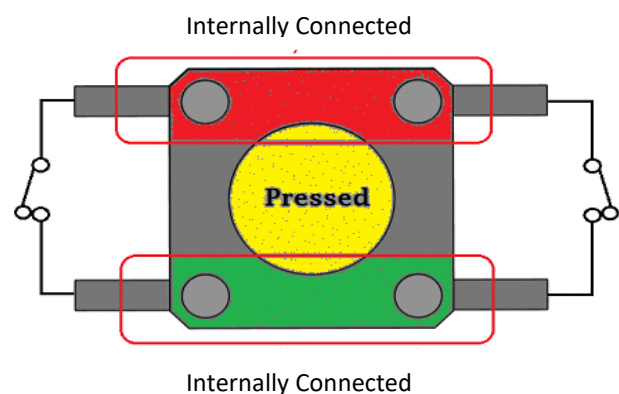


Figure 3.1 : Push Button Pinout

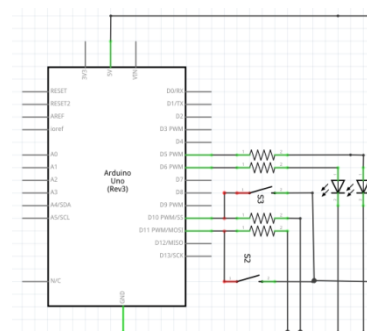


Figure 2.1 : Latch and Switch Button

```

// Set up variables
int switch1State = 0; // 0 means switch is not pressed
int switch2State = 0; // 1 means switch is pressed
int ledState = 0;     // 0 means LED is off
                     // 1 means LED is on

void setup(){

// Set the digital pin mode as input/output
pinMode(LED_RED , OUTPUT);

pinMode(LED_GREEN, OUTPUT);
pinMode(SWITCH_1, INPUT);
pinMode(SWITCH_2, INPUT);
// Initialize digital output pin state
digitalWrite(LED_RED , LOW);
}

void loop() {
switch1State = digitalRead(SWITCH_1); // Read switch 1 state
switch2State = digitalRead(SWITCH_2); // Read switch 2 state
if (switch1State == 0) { // Check if Switch 1 is pressed
while (digitalRead(SWITCH_1) == 0); // Wait until switch 1 is released
switch (ledState) {

case 0: digitalWrite(LED_RED , HIGH); // Turn on red LED
ledState = 1;
break;
case 1: digitalWrite(LED_RED , LOW); // Turn off red LED
ledState = 0;
break;
}}
if (switch2State == 0)
digitalWrite (LED_GREEN, HIGH); // Turn on green LED
if (switch2State == 1)
digitalWrite (LED_GREEN, LOW); // Turn off green LED
}

```

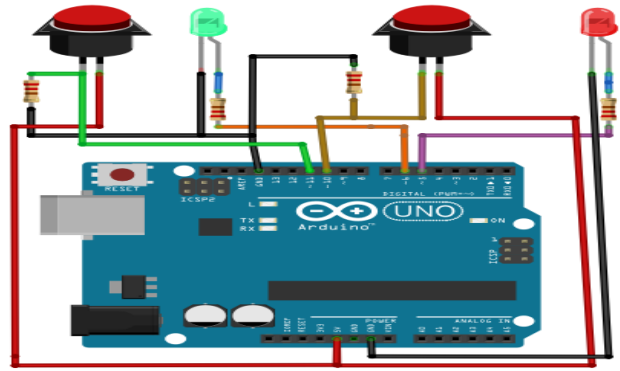


Figure 3.2 : Latch and Switch Button Pictorial Circuit Diagram

Discussion/Observation:

Conclusion :

Assessment Task :

1. Design a two way intersection with pedestrian walk cycle
2. Design a Two way Circuit switch

References :

1. <https://community.blynk.cc/t/2-way-lighting-circuit-control-with-blynk-and-feedback-updates/11102>

Experiment 4

Potentiometer Applications

Learning Outcomes

1. Use a potentiometer as a potential divider.
2. To read the analog voltage from the potentiometer and then use it to set the brightness of the external LED.
3. learn how to use software interrupts.

Working Principle

The basic principle of the potentiometer is that the potential drop across any section of the wire will be directly proportional to the length of the wire, provided the wire is of the uniform cross-sectional area and a uniform current flows through the wire.

The potentiometer is basically a long piece of uniform wire across which a standard cell is connected. In the actual design, the long wire is cut into several pieces and it is placed side by side and connected at the ends with a thick metal strip. The current flowing through the wire can be varied using a variable resistance (rheostat) connected to the circuit. The resistance can be changed manually for measuring the potential difference. The potential difference between any two points in a circuit is the amount of work done in bringing the charge from the first point to the second point. When there is a potential difference there will be a current flow in the circuit.

Part 1: Light dimmer using Potentiometer

Components List

1 Arduino Uno
1 10k ohms Potentiometer
1 Green Led
1 220 ohms Resistor
1 Bread Board
Connecting Wires



Figure 4.1 : Potentiometer Pinout

Circuit Construction

Connect the components by following the circuit diagram
.Remember that potentiometers have three pins.

Sketch

```
// Assign connection to Arduino Uno pins
const int LED = 5;
const int POT = A3; // Potentiometer output pin connection

// Set up variables
int potValue; // Potentiometer output value

void setup() {
```

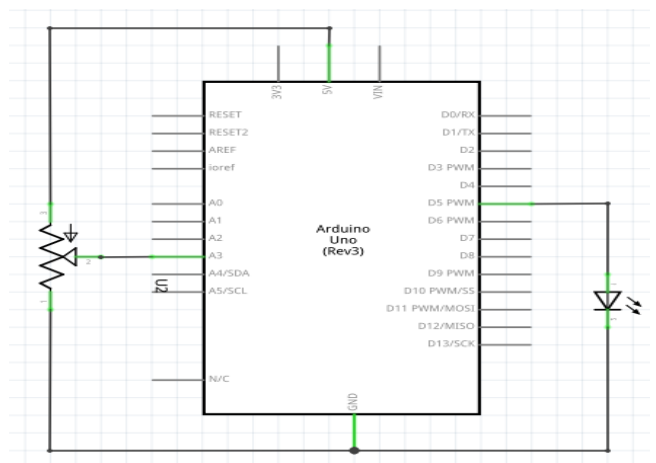


Figure 4.2 : Light dimmer Schematic Diagram

```
// Set the digital pin mode as input/output
pinMode(LED, OUTPUT);
pinMode(POT, INPUT);
}

void loop() {
  // Read and save analog value from potentiometer
  potValue = analogRead(POT);
  // Map potValue values 0 -1023 to 0-255 (PWM)
  potValue = map(potValue, 0, 1023, 0, 255);
  // Turn LED on with brightness set based on potValue
  analogWrite(LED, potValue);
  // Keep LED on at the given brightness for 100 ms
  delay(100);
}
```

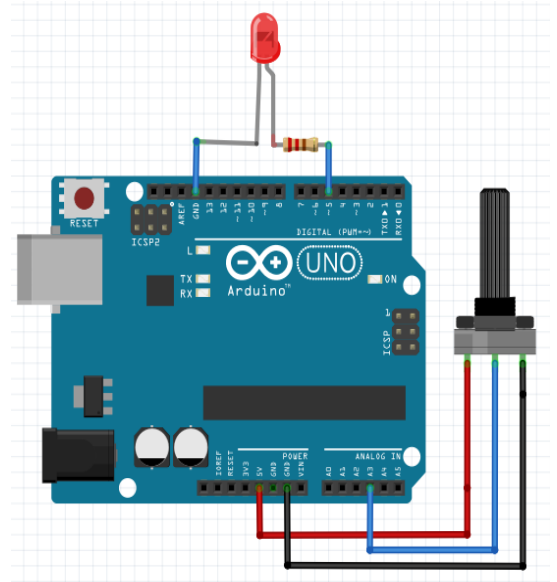


Figure 4.3 : Light dimmer Pictorial Circuit Diagram

Part 2 –Switching delay Using Potentiometer

List component

- 1 Arduino Uno
- 1 10k ohms Potentiometer
- 1 Green LED
- 1 220 ohms Resistor
- 1 Bread Board

Circuit Construction

Connect the components by following the circuit diagram .Remember that potentiometers have three pins.

Outer Pins are used for the Vcc (Power Source) and the other is ground .The middle pin (output) give us the variable of resistance value.

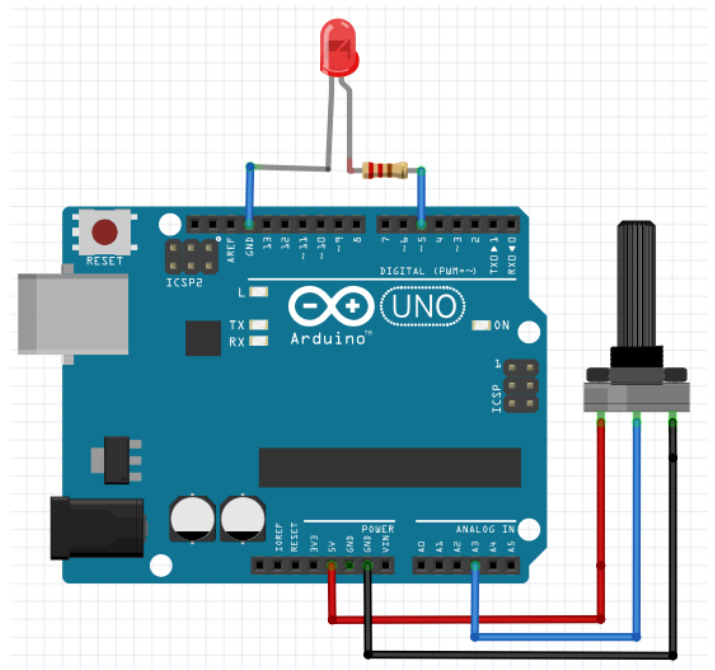


Figure 4.4 : Potentiometer value used as delay

Sketch

```
// Assign connection to Arduino Uno pins
const int POT = A3;
const int LED = 5;

// Set up variables
int potValue; // Potentiometer output value

void setup() {
  // Set the digital pin mode as output
  pinMode(LED, OUTPUT);
}
```

```
void loop() {  
  potValue = analogRead(POT); // Read and save analog value from potentiometer  
  digitalWrite(LED, LOW); // Turn LED off  
  delay(potValue); // Keep LED off by potValue ms  
  digitalWrite(LED, HIGH); // Turn LED on  
  delay(potValue); // Keep LED on by potValue ms  
}
```

Discussion/Observation:

Conclusion :

Assessment Task :

Design a Circuit to apply practically the Potentiometer to control the incandescent bulb using 220 V source (Note: Use Relay)

References :

<http://arduino.cc>

<http://www.ladyada.net/learn/arduino/index.html>

<http://learn.adafruit.com>

<https://kitronik.co.uk/blogs/resources/arduino-inventors-kit-exp-3>

Experiment 5

Controlling a Switch using LDR

Learning Outcome

1. Understand the working principle of LDR
2. Know how to Design a circuit to Control Load using LDR
3. Turn LED ON and OFF through LDR

List of Components

1 Arduino Uno
1 Light Dependent Resistor(LDR)
1 Red LED
1 220 ohms Resistor, R1
1 1K Ω Resistor, R2
1 Bread Board
Connecting Wires

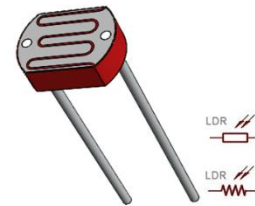


Figure 5.1 :LDR Pinout

Light Dependent Resistor

LDR is **Light Dependent Resistor**. LDRs are made from semiconductor materials to enable them to have their light-sensitive properties. There are many types but one material is popular and it is cadmium sulfide (CdS). These LDRs or PHOTO RESISTORS works on the principle of “Photo Conductivity”. Now what this principle says is, whenever light falls on the surface of the LDR (in this case) the conductance of the element increases or in other words, the resistance of the LDR falls when the light falls on the surface of the LDR. This property of the decrease in resistance for the LDR is achieved because it is a property of semiconductor material used on the surface.

Working of LDR controlled LED using Arduino

As per the circuit diagram, we have made a voltage divider circuit using LDR and 100k resistor. The voltage divider output is feed to the analog pin of the Arduino. The analog Pin senses the voltage and gives some analog value to Arduino. The analog value changes according to the resistance of LDR. So, as the light falls on the LDR the resistance of it get decreased and hence the voltage value increase.

Sketch

```
// Assign connection to Arduino Uno pins
const int LDR = A3;
const int LED = 5;

void setup() {
  Serial.begin(9600); // Begin serial communication
  //Set the baud rate at 9600
  pinMode( LED, OUTPUT ); // Set the digital pin mode as output
  // the loop is where your program runs repeatedly.
}

void loop()
```

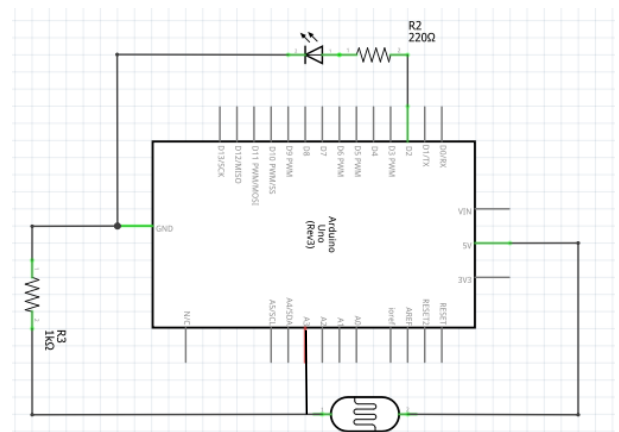


Figure 5.2. : LDR Switching Schematic Diagram

```

{
  int val = 0;    // variable declaration from the resistance of LDR

  val = analogRead(LDR); // read the value from the sensor

  //display the ldr resistance value in serial monitor..

  Serial.println(analogRead(LDR));

  digitalWrite(LED, HIGH); // Turn LED on

  delay(val); // Keep LED on for val ms

  digitalWrite(LED, LOW); // Turn LED off

  delayMicroseconds(100); // Keep LED off for val microseconds
}

```

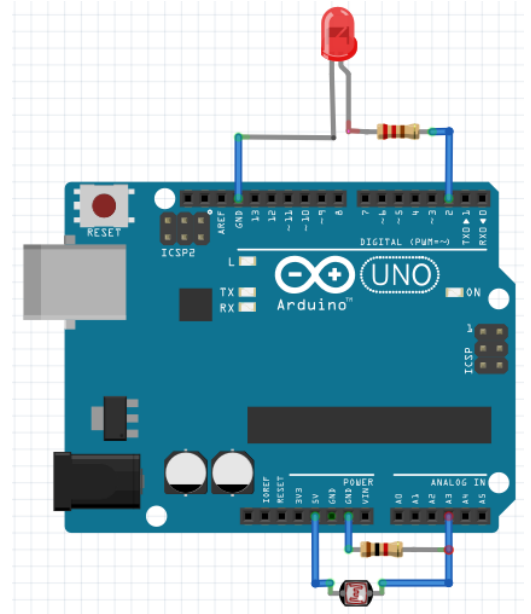


Figure 5.3. : LDR Switching Pictorial Circuit Diagram

Discussion/Observation:

Conclusion :

Assessment Task :

Design a Circuit to apply practically the LDR to control the incandescent bulb using 220 V source (Note: Use Relay) in application to Street Light.

References:

<http://arduino.cc>

<http://www.ladyada.net/learn/arduino/index.html>

<http://learn.adafruit.com>

Experiment 6

Piezo speakers (buzzer)

Learning Outcomes

1. learn how to use a buzzer or piezo speaker with Arduino
2. Learn to interface and program with different output.

List of Components :

1 Arduino uno
1 Breadboard
1 Buzzer / piezo speaker
1 220 ohms Resistor
1 Transistor NPN 2N222



Figure 6.1. : Buzzer Pinout

Sketch

```
// Assign connection to Arduino Uno pins  
const int BUZZER = 3; // buzzer to pin 3  
  
void setup() {  
  
  pinMode(BUZZER, OUTPUT); // Set pin 3 as an output  
  
}  
  
void loop() {  
  tone(BUZZER, 1000); // Sound buzzer at 1KHz frequency signal  
  delay(1000); // Keep sound on for 1 sec  
  noTone(BUZZER); // Stop buzzer sound  
  delay(1000); // Keep sound off for 1 sec  
  
}
```

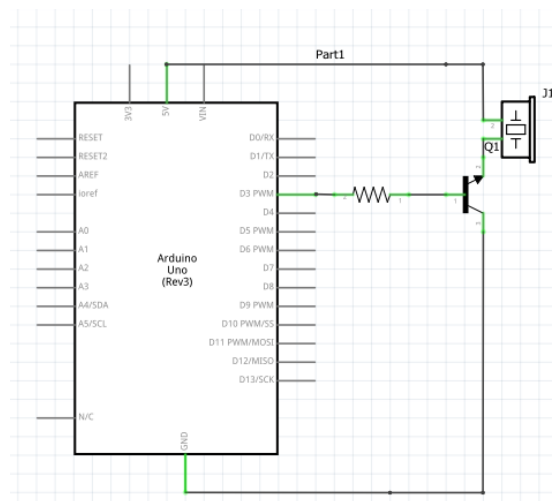


Figure 6.2. : LDR Switching Schematic Diagram

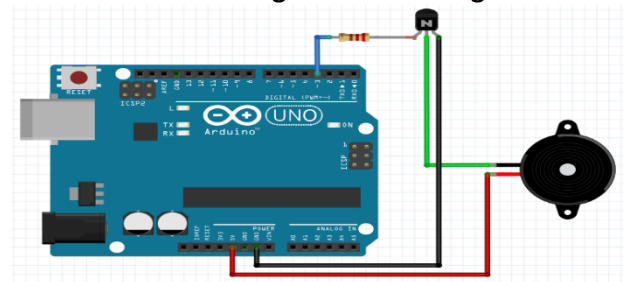


Figure 6.3 : LDR Switching Wiring Diagram

Discussion/Observation:

Conclusion :

Assessment.

- 1. Interface any possible output and use digital output pin.**
- 2. Interface any possible output and use Analog pin.**

References :

<http://arduino.cc>

<http://www.ladyada.net/learn/arduino/index.html>

<http://learn.adafruit.com>

Experiment 7

Displaying Temperature in Serial Monitor

Learning Outcomes

1. Learned to interface the LM 35 in Arduino
2. Learn to use Datasheet
3. Know how to use serial interface in Arduino.

Principle of Activity

This Activity is a simple project which can be use as a thermometer. It can sense the temperature it is put around and transmit it to degrees Celsius. It consists of very little parts which is cheap and reliable. This project consists of a LM35 Temperature Sensor which is the most often used and the output /Reading will be displayed in the monitor(Serial Interface)

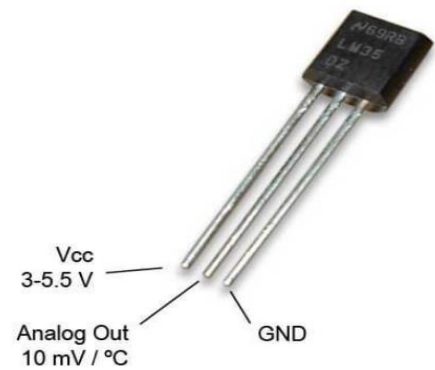


Figure 7.1. : LM35 Pinout

Components List

- 1 Arduino uno
- 1 Breadboard
- 1 LM35
- 1 Breadboard

Sketch

// Assign connection to Arduino Uno pin

const int TEMP_SENSOR = A2;

// Set up variable
float temperature;

```
void setup() {  
  Serial.begin(9600); // Set the baud rate at 9600 and  
                      // Begin serial communication  
  pinMode(TEMP_SENSOR, INPUT); // Set the digital pin mode as output  
}
```

```
void loop() {  
  temperature = analogRead(TEMP_SENSOR);
```

Diagram

```
Serial.print(temperature); // Display temperature in serial monitor.  
delay(1000); // Pause for 1 second before next temperature read  
}
```

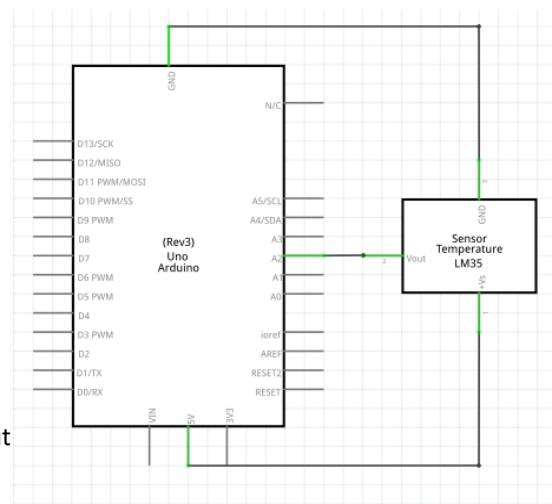


Figure 7.2 : Temperature Sensor in Arduino Circuit

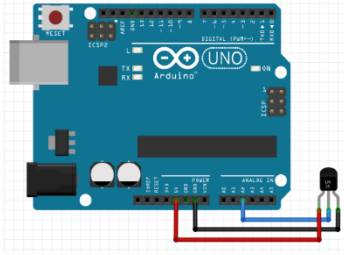


Figure 7.2 : Temperature Sensor Pictorial Circuit Diagram

Discussion/Observation:

Conclusion :

Assessment : Design a circuit that it will read and display the temperature and humidity of the surrounding and display in LCD and Desktop (Screen)

References :

<https://create.arduino.cc/projecthub/infoelectorials/project-003-arduino-lm35-temperature-sensor-project-0a43ba>

<https://www.circuitstoday.com/lm35-and-arduino-interfacing>

Experiment 8

Using Relay Module in Arduino

Learning Outcomes :

- Know how to read NO and NC in Relay
- Learn to interface relay with different loads

Principle of the Activity

A relay is an electrically operated switch that can be turned on or off, letting the current go through or not, and can be controlled with low voltages, like the 5V provided by the Arduino pins.

Controlling a relay module with the Arduino is as simple as controlling any other output as we'll see later on.

This relay module has two channels (those blue cubes). There are other models with one, four and eight channels. This module should be powered with 5V, which is appropriate to use with an Arduino. There are other relay modules that are powered using 3.3V, which is ideal for ESP32, ESP8266, and other microcontrollers.

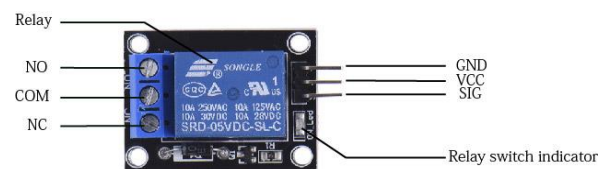


Figure 8. 1 Relay Pinout

Components List

- 1 Arduino uno
- 1 Breadboard
- 1 Relay Module(5 Volts)
- 1 LED (red)

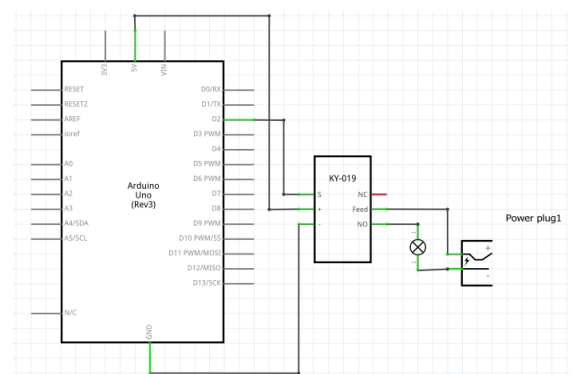
Sketch

```
// Assign connection to Arduino Uno pins
const int RELAY = 2;

void setup() {
  // Initialize digital output pins state
  pinMode(RELAY, OUTPUT);
}

void loop() {
  digitalWrite(RELAY, HIGH); // Turn the
  relay ON
  delay(1000); // delay in 1sec
  digitalWrite(RELAY, LOW); // turn the relay
  OFF
  delay(1000);
}
```

ng Schematic Diagram



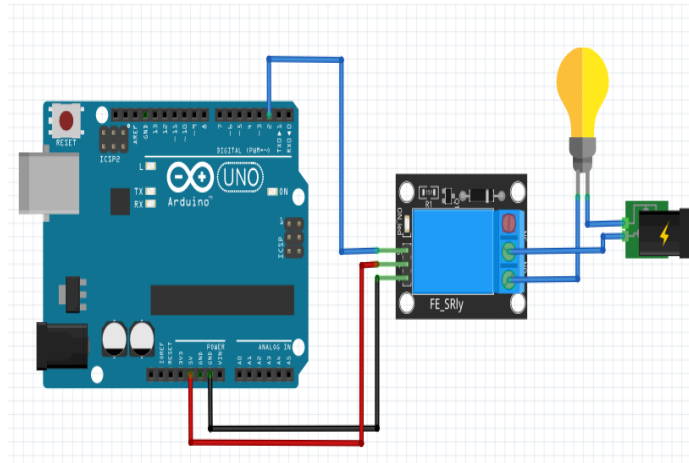


Figure 8.3 Relay Switching Pictorial Circuit Diagram

Discussion/Observation:

Conclusion :

Assessment Task : Design a Circuit that will read data in sensor and it will display output

References :

1. <https://create.arduino.cc/projecthub/muhammed-shameel-k-v/how-to-use-a-relay-with-arduino-e04e3c>

2. <https://randomnerdtutorials.com/guide-for-relay-module-with-arduino/#:~:text=A%20relay%20is%20an%20electrically,we'll%20see%20later%20on.>

Experiment 9

Controlling DC Motor

Learning Outcome

1. Learn how to interface and control DC motor .
2. Learn to apply Darlington transistor as Current amplifier.

Principle of the Activity

A direct current, or DC, motor is the most common type of motor. DC motors normally have just two leads, one positive and one negative. If you connect these two leads directly to a battery, the motor will rotate. If you switch the leads, the motor will rotate in the opposite direction. To control the direction of the spin of DC motor, without changing the way that the leads are connected, you can use a circuit called an H-Bridge. An H bridge is an electronic circuit that can drive the motor in both directions. H-bridges are used in many different applications, one of the most common being to control motors in robots. It is called an H-bridge because it uses four transistors connected in such a way that the schematic diagram looks like an "H." You can use discrete transistors to make this circuit, but for this tutorial, we will be using the L298 H-Bridge IC. The L298 can control the speed and direction of DC motors and stepper motors and can control two motors simultaneously. Its current rating is 2A for each motor. At these currents, however, you will need to use heat sinks.

List of Components :

- 1 Arduino uno
- 1 DC Motor (5V)
- 1 TIP 120
- 1 10K Ohm Resistor
- 2 Breadboard
- 1 Bridge type IC (Optional)
- Connecting Wires

Sketch

```
// Assign connection to Arduino Uno pin
const int DC_MOTOR = 5;

void setup() {
  pinMode(DC_MOTOR, OUTPUT); // Set the
  digital pin as output
  Serial.begin(9600); // Begin serial
  communication // Set the baud rate at 9600
  Serial.println("Type 0 to 255 for the speed of DC
  motor ");
}

void loop() {
  if (Serial.available()) {
    int motorSpeed = Serial.parseInt();
    if (motorSpeed >= 0 && motorSpeed <= 255)
      analogWrite(DC_MOTOR, motorSpeed);
    delay(1000); //let the motor run for at least
    1 second
  }
}
```

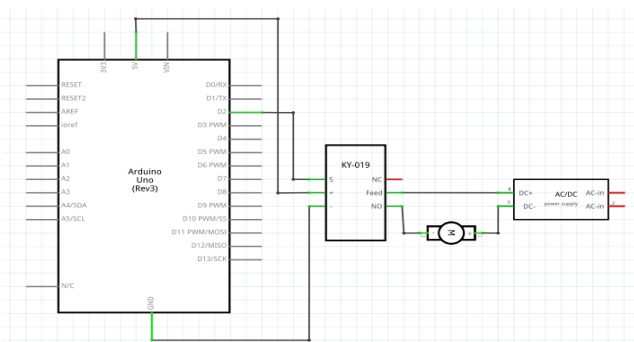


Figure 9.1 DC motor in Arduino Schematic Diagram

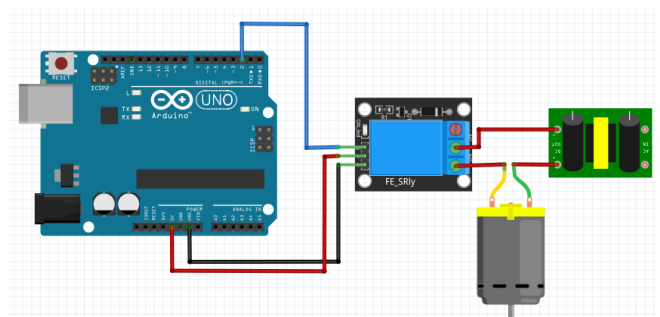


Figure 9.2 DC motor Pictorial Circuit Diagram

Discussion/Observation:

Conclusion :

Assessment Task :

Design /program that will allow to forward and reverse the rotation of the motor.

References :

<https://www.arduino.cc/>

Experiment 10

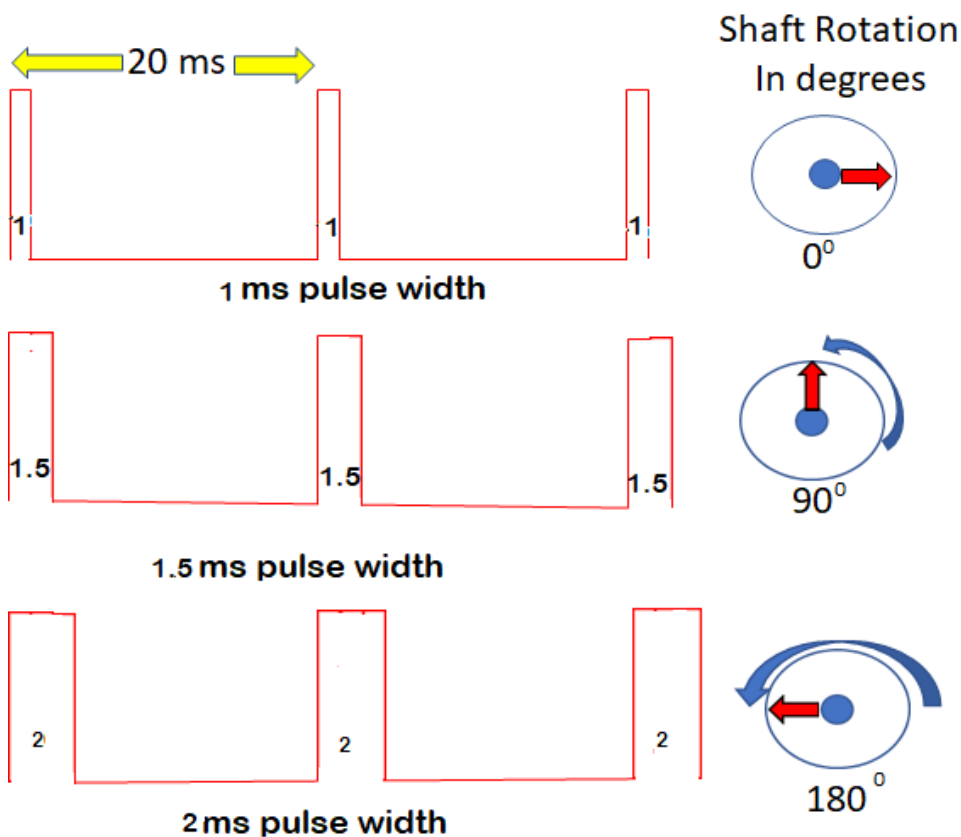
Control of Servo Motor

Learning Outcomes

1. learn how to control a servo motor using an Arduino
2. Identify Servo Motor Pins

Principle of Activity

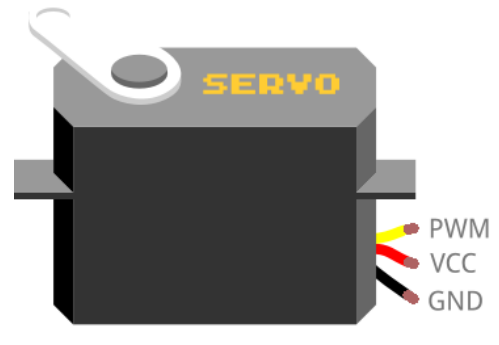
Servos are DC motors that have been geared down to reduce the speed and increase the torque of the motor. They also have built in circuits that control the angle of rotation one degree at a time, and hold that position until another input is received. Servos will rotate a certain number of degrees depending on the width of the electrical pulses delivered by the Arduino:



The servo expects one pulse every 20 ms. For most servos, a 1 ms pulse results in a zero degree rotation, a 1.5 ms pulse results in a 90 degree rotation, and a 2 ms pulse results in a 180 degree rotation.

List of Components :

- 1 Arduino uno
- 1 Breadboard
- 1 Servo Motor
- 1 10K Ω Potentiometer



Sketch

Figure 10.1 Servo motor Pinout

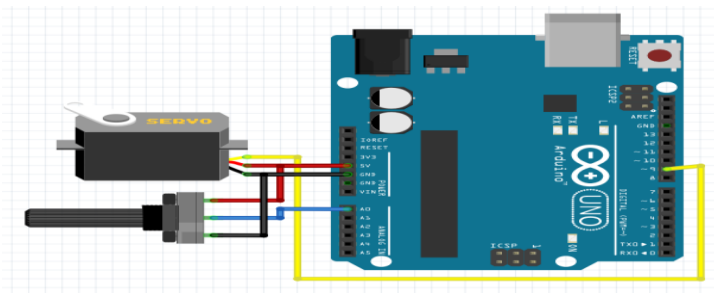
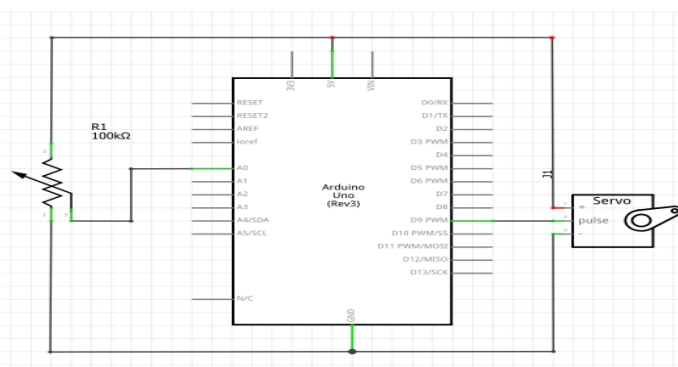


Figure 10.2 Servo motor in Arduino Pictorial Circuit Diagram



```
#include <Servo.h>
// Assign connection to Arduino Uno pin
const int SERVO =9;
const int POTENTIOMETER= A0;
// Set up variable
int pos = 0; // variable to store the servo position
Servo myservo; // Create a servo object to control a servo
```



```
        // Eight servo objects, max, can be created
void setup() {
  pinMode(POTENTIOMETER,INPUT);
  myservo.attach(SERVO);
  // Attach pin SERVO (pin 9) to servo object
}
void loop() {
  int potValue = analogRead(A0);
  potValue = map (potValue, 0, 1023, 0, 180);
  myservo.write(potValue);
}
```

Discussion/Observation:

Conclusion :

Assessment :

The servo motor should move to 0 degrees, pause for one second, then move to 90 degrees, pause for one second, then move to 180 degrees, pause for one second, then start over.

References :

Experiment 11

DISPLAY MESSAGE IN LCD

Learning Outcomes

1. Learn to interface LCD to Arduino
2. Learning to identify PINOUT of LCD

Principle of Activity

What is LCD Character Display?

An LCD character display is a unique type of display that can only output individual ASCII characters with fixed size. Using these individual characters then we can form a text. If we take a closer look at the display we can notice that there are small rectangular areas composed of 5×8 pixels grid. Each pixel can light up individually, and so we can generate characters within each grid. The number of the rectangular areas define the size of the LCD. The most popular LCD is the 16×2 LCD, which has two rows with 16 rectangular areas or characters. Of course, there are other sizes like 16×1, 16×4, 20×4 and so on, but they all work on the same principle. Also, these LCDs can have different background and text color.

16×2 LCD Pinout

It has 16 pins and the first one from left to right is the **Ground** pin. The second pin is the **VCC** which we connect the 5 volts pin on the Arduino Board. Next is the **Vo** pin on which we can attach a potentiometer for controlling the contrast of the display.

Next, The **RS** pin or register select pin is used for selecting whether we will send commands or data to the LCD. For example if the RS pin is set on low state or zero volts, then we are sending commands to the LCD like: set the cursor to a specific location, clear the display, turn off the display and so on. And when RS pin is set on High state or 5 volts we are sending data or characters to the LCD.

Next comes the **R/W** pin which selects the mode whether we will read or write to the LCD. Here the write mode is obvious and it is used for writing or sending commands and data to the LCD. The read mode is used by the LCD itself when executing the program which we don't have a need to discuss about it in this tutorial.

Next is the **E** pin which enables the writing to the registers, or the next 8 data pins from D0 to D7. So through this pins we are sending the 8 bits data when we are writing to the registers or for example if we want to see the latter uppercase A on the display we will send 0100 0001 to the registers according to the ASCII table. The last two pins **A** and **K**, or anode and cathode are for the LED back light.

Schematic Diagram

Components List

1 Arduino uno
 1 LCD (2x16)
 1 10k Potentiometer-
 1 Resistor=220 ohms
 Jumper Wires
 Breadboard

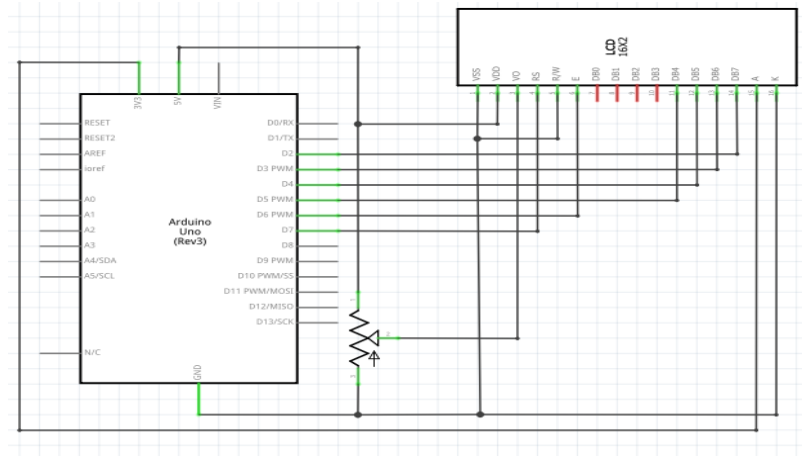


Figure 11.2 LCD Arduino Display Schematic Circuit

LCD Pinout

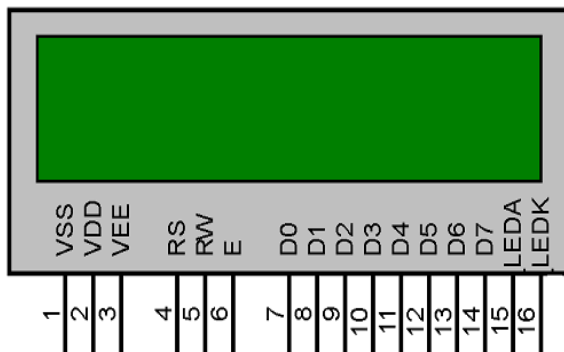


Figure 11.1 LCD (16x2) Pinout Diagram

Circuit Construction

LCD Wire Connections:

LCD to Arduino Uno
 RS ==digital pin 7
 Enable== digital pin 6
 D4 == pin 5
 D5 ==I pin 4
 D6 == pin 3
 D7 ==I pin 2
 Potentiometer

Outer Pin- Ground and Source
 Middle Pin to Pin 3 of LCD

Sketch

```
// Include library used for LCD
#include <LiquidCrystal.h>

// Create LiquidCrystal object mylcd and
// Assign connection to Arduino Uno pins
LiquidCrystal mylcd(7,6,5,4,3,2);

void setup() {
  // Initialize the LCD screen interface and
  // Specify the width and height of the display
  mylcd.begin(16, 2);
  // Display message to LCD screen
  mylcd.print("My Program!");
}

void loop() {
  // Set the cursor to column 0, second line 1
  mylcd.setCursor(0, 1);
  // Print number of seconds since reset
  mylcd.print(millis()/1000);
}
```

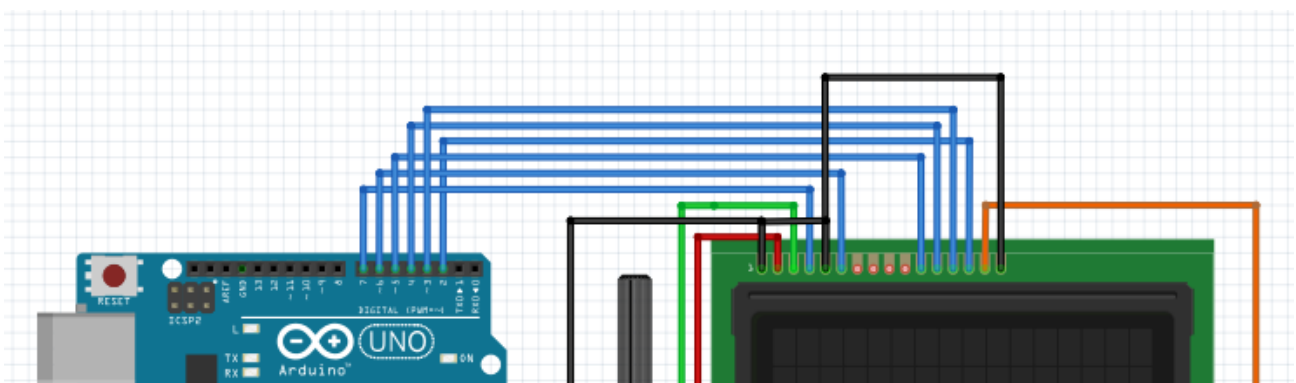


Figure 11.3 LCD Arduino Display Pictorial

Discussion/Observation:

Conclusion :

Assessment Task :

Design/Program that will display your Given Name , Middle Initial and Family name in 3 rows

References :

<https://www.circuitbasics.com/how-to-set-up-an-lcd-display-on-an-arduino/>

Experiment 12

Security System using Keypad with LCD

Learning Outcomes

1. Learn to Interface keypad in Arduino
2. Learn to interface and program both keypad and lcd in Arduino

Learning Principle/Project

In this experiment , The Password-Based Security System Using Arduino & Keypad. As thefts are increasing day by day security is becoming a major concern nowadays. So a digital code lock can secure your home or locker easily. It will open your door only when the right password is entered.

The circuit of this project is very simple which contains Arduino, keypad module, buzzer, Servo Motor, and LCD. Arduino controls the complete processes like taking a password from the keypad module, comparing passwords, driving buzzer, rotating servo motor, and sending status to the LCD display. The keypad is used for taking the password. The buzzer is used for indications. Servo motor is used for opening the gate while rotating and LCD is used for displaying status or messages on it.

List of Components

Components List

1 Arduino Uno
1 Keypad-4x3
1 LCD -2x16
1 10K Potentiometer
1 220-ohm Resistor-
1 Door Lock Solenoid
1 Transistor or relay module
Jumper Wires
Breadboard

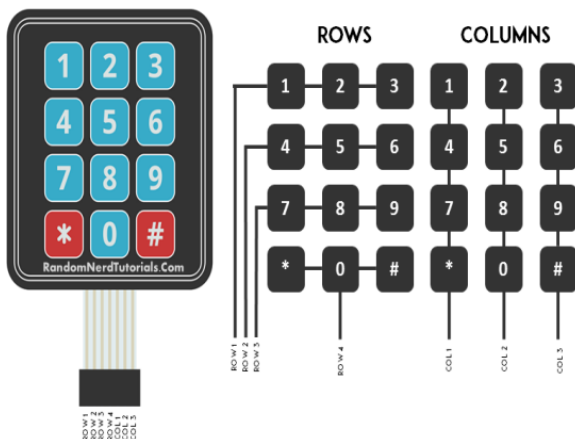
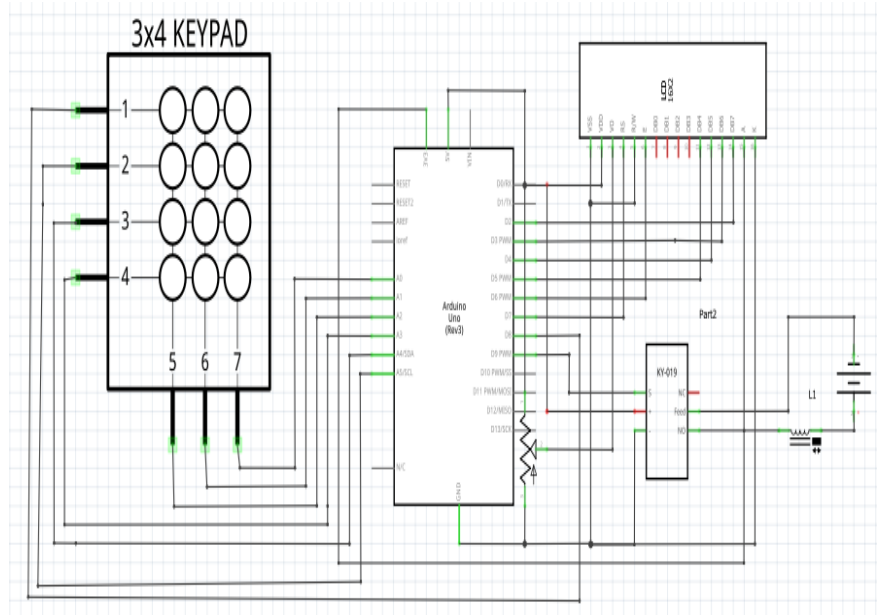


Figure 12.2 Security System using keypad Schematic Diagram

Figure 12.1 Keypad 4x3 Pinout

Circuit Diagram

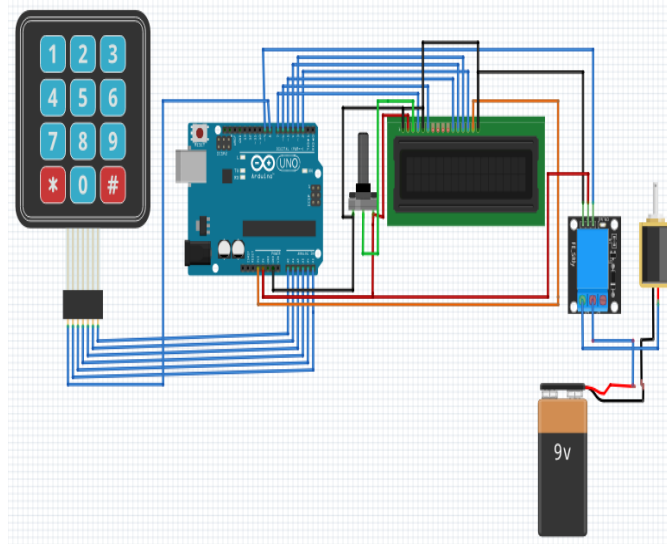


Figure 12.3 Security System using keypad Pictorial Circuit Diagram

Sketch

```
// Include libraries used for LCD and Keypad
#include <Keypad.h>
#include <LiquidCrystal.h>
// Create LiquidCrystal object mylcd
// with parameters (rs, en, d4, d5, d6, d7), and
// Assign connection to Arduino Uno pins
LiquidCrystal mylcd(7,6,5,4,3,2);
// Set up variables
char* myPassword = "110880";
int position = 0;
const byte rows = 4;
const byte cols = 3;
char keys[rows][cols] = {
  {'1','2','3'},
  {'4','5','6'},
  {'7','8','9'},
  {'*','0','#'}};
byte rowPins[rows] = {8,A1,A2,A3};
byte colPins[cols] = {A4,A5,A6};
// Create Keypad object keypad and
// Assign connection to Arduino Uno pins
Keypad keypad = Keypad(makeKeymap(keys),
```

```
rowPins, colPins,
rows, cols);
// Assign door lock connection to Arduino Uno pin
int doorLock = 2;
void setup()
{
  pinMode(doorLock, OUTPUT); // Set the digital pin as output
  mylcd.begin(16, 2); // Initialize the LCD screen interface
  and
  // Specify the width and height of the display
  setLocked(true);
}
void loop()
{
  char key = keypad.getKey(); // Get a char entered through
  keypad
  if (key == '*' || key == '#') // Keys * or # resets password
  reading
  {
    position = 0; // Next char read is for first char of password
    setLocked(true); // Lock door
  }
  // Check if char input at the given position in myPassword is
  correct
```

```

// If char entered is wrong next, char read is for the same
position
if (key == myPassword[position]) {
    position++; // Next char read is for new position in
myPassword
}
if (position == 6) { // Required password is correctly entered
    setLocked(false); // Unlock door
}
delay(1000);
}
void setLocked(int locked)
{
    if (locked) {

```

```

        digitalWrite(doorLock, HIGH); // Set output pin HIGH to
lock door
        mylcd.print("DOOR LOCKED");// Print a message to LCD
screen
    }
    else {
        digitalWrite(doorLock, LOW);// Set output pin HIGH to lock
door
        mylcd.print("DOOR OPEN"); // Print a message to LCD
screen
    }
}
}

```

Discussion/Observation:

Conclusion :

Assessment Task : Design A security system that will automatically lock the door and send message to the owner if there are attempt to change the password .

References :

Experiment 13

Ultrasonic Sensor Applications

Learning Outcome

- 1. Learn to interface ultrasonic sensor in Arduino and know its different applications**

Learning Principle

Ultrasonic sensors work by sending out a sound wave at a frequency above the range of human hearing. The transducer of the sensor acts as a microphone to receive and send the ultrasonic sound. Our ultrasonic sensors, like many others, use a single transducer to send a pulse and to receive the echo. The sensor determines the distance to a target by measuring time lapses between the sending and receiving of the ultrasonic pulse.

The working principle of this module is simple. It sends an ultrasonic pulse out at 40kHz which travels through the air and if there is an obstacle or object, it will bounce back to the sensor. By calculating the travel time and the speed of sound, the distance can be calculated.

Ultrasonic sensors are a great solution for the detection of clear objects. For liquid level measurement, applications that use infrared sensors, for instance, struggle with this particular use case because of target translucence.

For presence detection, ultrasonic sensors detect objects regardless of the color, surface, or material (unless the material is very soft like wool, as it would absorb sound.)

To detect transparent and other items where optical technologies may fail, ultrasonic sensors are a reliable choice.

Part 1: Obstacle Warning System

Circuit Diagram

List of Components

- 1 Arduino Uno
- 1 Ultrasonic Sensor



1 Piezo Buzzer
 1 Relay Module
 1 1K Ohm Resistor
 BreadBoard

Sketch

Figure 13.1: Ultrasonic HC-SR04 Pinout

```
int const trigger = 3;

int const echo = 2;
int const buzzer = 4;
void setup()
{
  pinMode(trigger, OUTPUT);
  // trig pin will emit pulses output
  pinMode(echo, INPUT);
  // echo is a pin as input to received signal
  pinMode(buzzer, OUTPUT);
  // buzzer set as output
}
void loop()
{
  digitalWrite(trigger, HIGH); //send pulses
  delay(1);
  digitalWrite(trigger, LOW);
  //measuring the duration from trigger to echo .
  int duration = pulseIn(echo, HIGH);
  // distane is divided to 2 since its transmitting and receiving.
  int distance = (duration/2) / 29.1;
  // if distance less than 40cm and more than 0
  if (distance <= 40 && distance >= 0) {
    // Buzzer will be high and it sounds
    digitalWrite(buzzer, HIGH);
  } else {
    // Buzzer will be low , no sounds
    digitalWrite(buzzer, LOW);
  }

  delay(20);
}
```

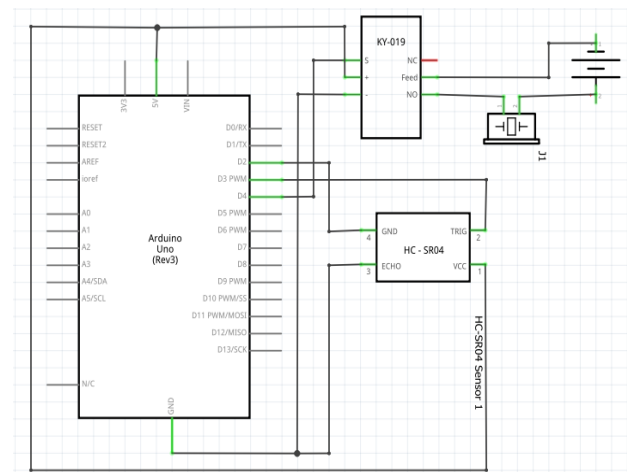


Figure 13.2: Ultrasonic Schematic Diagram

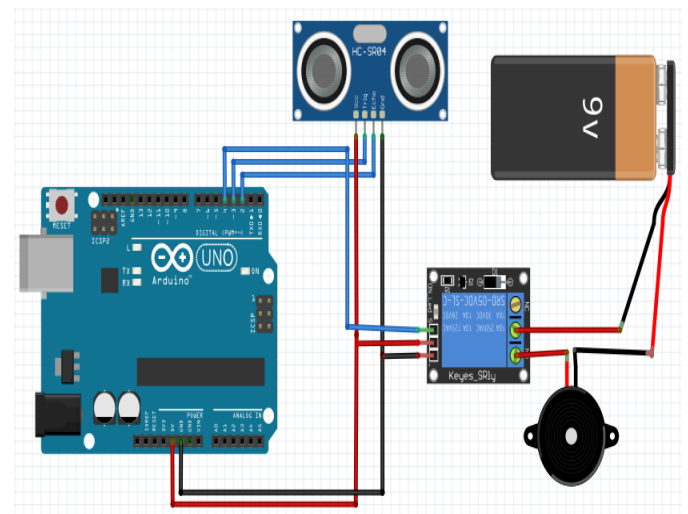


Figure 13.3: Ultrasonic sensor with Buzzer Pictorial Circuit Diagram

Discussion/Observation:

Conclusion :

References :

Experiment 14

Distance Measurement

Learning Outcomes

Components List

- 1 Arduino Uno
- 1 LCD 16x2
- 1 220 Ohm Resistor
- 1 Rotary Potentiometer - 10k Ohm, Linear
- 1 Ultrasonic Sensor - HC-SR04
- BreadBoard
- Connecting Wires Pack - M/M

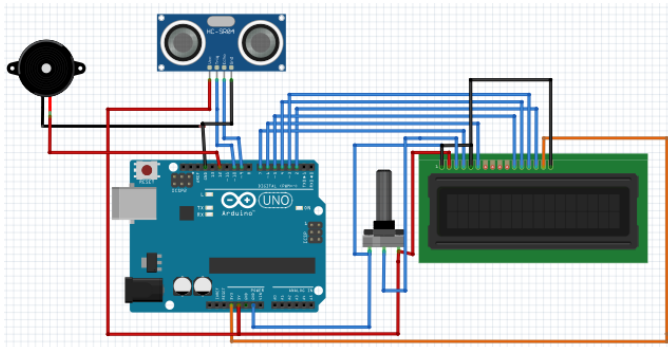


Figure 13.1 Ultrasonic Sensor with

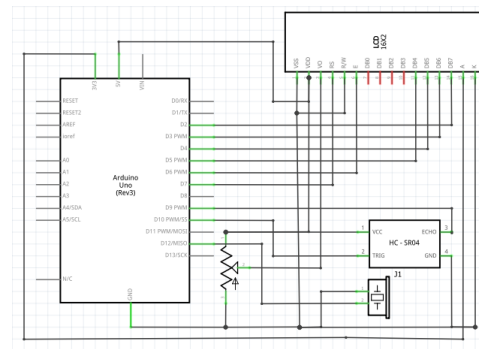


Figure 13.3 Ultrasonic Sensor with Display Schematic Diagram

Display Pictorial Circuit Diagram

Sketch

```
// Assign connection to Arduino Uno pins
const int TRIGGER= 10;
const int ECHO= 9;
const int BUZZER = 2;
// Set up variables
int duration; // Travel duration of ultrasonic signal from sensor
// to object and back
int distance; // Distance of object from sensor

void setup() {
  // Set TRIGGER pin as output
  // sensor send out ultrasonic signal
  pinMode(TRIGGER, OUTPUT);
  pinMode(ECHO, INPUT); // Set ECHO pin as input
  // ultrasonic signal echo
  pinMode(BUZZER, OUTPUT); // Set BUZZER set as output
  digitalWrite(BUZZER, LOW); // Buzzer is set to off initially
}

void loop() {
  // Sense distance of object from sensor
  // TRIGGER pin initially set LOW
  digitalWrite(TRIGGER, LOW);
  delayMicroseconds(2); // for 2 microseconds
  // Sensor sends out ultrasonic pulses
  digitalWrite(TRIGGER, HIGH);
  delayMicroseconds(5); // for 5 microseconds
  digitalWrite(TRIGGER, LOW); // Sensor stops emitting pulses
  // is sent up to when its echo is received
  // Time in microseconds from when signal
  duration = pulseIn(ECHO, HIGH);
  // Distance of object from sensor in cm
  // is half of round-trip distance traveled by sound
  // Sound speed = 29 centimeters/microsecond
  distance = duration / 2 / 29.0 ;

  // If object is at or within 40cm, the buzzer sounds
  if (distance <= 40 && distance >= 0) {
    digitalWrite(BUZZER, HIGH);
    delay(200); // Buzzer sounds for 200 ms
    digitalWrite(BUZZER, LOW);
    delay(200); // Buzzer is quite for at least 200 ms
  }
}
```

Discussion/Observation:

Conclusion :

References :

Experiment 15

Temperature and Humidity Measurement

Learning Outcomes

1. Learn to interface temperature and humidity sensor.
2. Learn to display the output in LCD and Desktop PC
3. Know how to identify the different types of Temperature and humidity sensors

Principle of The activity

These sensors have been designed for various applications to measure the humidity as well as the temperature of the environment. They do so by finding the amount of water vapor present in the air around the sensors. The amount of moisture in the gas can be a mixture of different elements including nitrogen, water vapor, argon, etc. Since the humidity can have huge effects on different biological, chemical, and physical processes, it should be measured and controlled in different industries and hence, there's a need for these sensors to help us out.

Temperature and humidity sensors have two different ways of collecting data and measuring humidity and temperature. One type measures Relative Humidity (also known as RH) and the other type measures Absolute Humidity (also known as AH). They can also be categorized based on their size. Small sensors are used for smaller purposes and larger ones are usually used for industrial applications.

These sensors have capacitive humidity sensing elements as well as a thermistor which is used to sense the environment's temperature. There are two electrodes in the humidity sensing element (capacitor) and a moisture holding substrate works as a dielectric between these two electrodes. Whenever there is a change in the humidity levels, changes occur to the capacitance value accordingly. There's an integrated IC in the unit that receives the

measured data and processes the resistance values that have been changed due to the change in humidity, and converts the data into a digital form for the readers. So, this is how digital temperature and humidity sensors such as DHT11 measure humidity. But how about the temperature?

Humidity and temperature sensor applications

As we mentioned above, these sensors can be found in many devices and they have a wide variety of applications! They can even help the patients who have trouble breathing by enabling them to keep the humidity and the temperature of the place at the optimum level. To predict weather conditions, weather stations use these sensors too. They can be used in heating systems, ventilation systems and even air conditioning systems. These sensors can also be used for greenhouses where the humidity values should be checked constantly. Museums can benefit from them as well since the artifacts and the objects in these places should be kept under certain conditions.

Now you know all there's to know about temperature and humidity sensors and their different models. Have you ever worked with these sensors directly? Comment below and let us know what you think about them and share your experience with us. Plus, feel free to sign up on Linquip to ask any questions you have from our experts who have years of experience in this industry.

Components List

- 1 LCD 16x2
- 1 220 Ohm Resistor
- 1 Rotary Potentiometer - 10k Ohm, Linear
- 1 DHT22/11 Humidity and Temperature Sensor
- 1 10K Ohm Resistor
- BreadBoard
- Connecting Wires Pack - M/M



Figure 15. 1 : DHT Pinout

Sketch

```
// Include libraries used for LCD and DHT Sensor
#include <dht.h>
#include <LiquidCrystal.h>

// Create LiquidCrystal object mylcd
// with parameters (rs, en, d4, d5, d6, d7), and
// Assign connection to Arduino Uno pins
LiquidCrystal mylcd(8, 7, 3, 4, 5, 6);

// Create dht object mydht
```

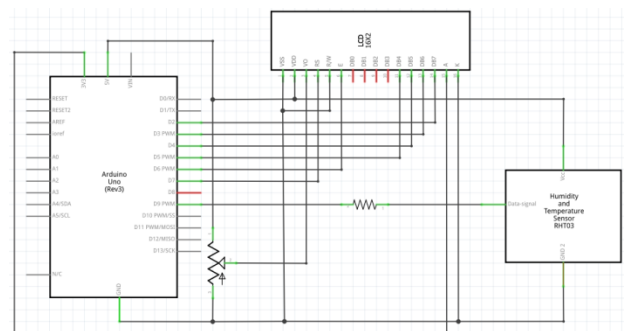


Figure 15.2 Temperature and Humidity Sensor Schematic Diagram


```

dht mydht;

// Assign connection to Arduino Uno pin
const int DHT_PIN = 2;
void setup() {
  // Initialize the LCD screen interface and
  // Specify the width and height of the display
  mylcd.begin(16, 2);
  Serial.begin(9600); // Set the baud rate at 9600 and
                     // Begin serial communication
}

```

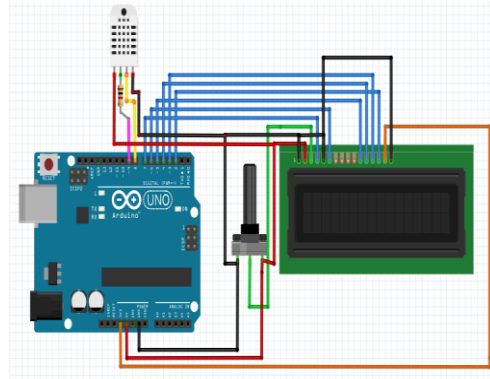


Figure 15.3 Temperature and Humidity Sensor with pictorial Diagram

```

void loop() {

// Read humidity and temperature data from sensor
mydht.read11(DHT_PIN);
// Display temperature
mylcd.setCursor(0,0);
mylcd.print("Temp: ");
mylcd.print(mydht.temperature);
mylcd.print("C");
// Display humidity
mylcd.print("Humid: ");
mylcd.print(mydht.humidity);
// Wait for 2 seconds before reading next data
delay(2000);
}

```

Discussion/Observation:

Conclusion :

References :

Experiment 16 Automated Irrigation System

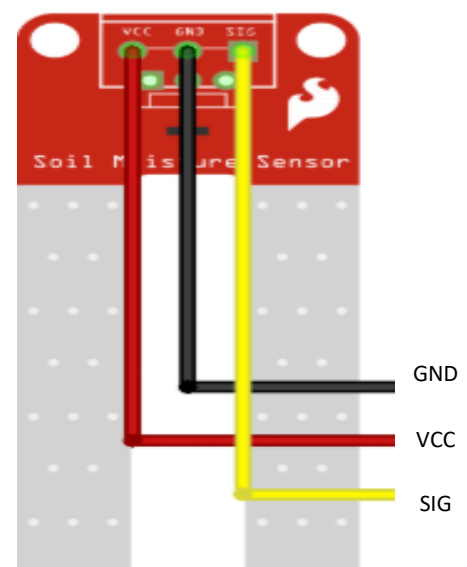
Learning Outcome

1. Learn how to interface Soil moisture sensor
 - 2.** Learn to program based on readings of the Sensor
- Principle of The activity :

The Soil Moisture Sensor **uses capacitance to measure dielectric permittivity of the surrounding medium**. In soil, dielectric permittivity is a function of the water content. The sensor creates a voltage proportional to the dielectric permittivity, and therefore the water content of the soil.

Components List

- 1 Arduino
- 1 Soil Moisture sensor
- 1 Pump (5VDC or 12 V DC)
- 1 Relay or Transistor
- 1 Breadboard



Sketch

```
// Assign connection to Arduino Uno pins
const int PUMP = 5;
const int SOIL_MOISTURE_SENSOR = A3;
// Set up variable
const int MOISTURE_LEVEL= 500;
int soil_moisture;

void setup() {
  pinMode(PUMP,OUTPUT); // Set the PUMP pin as output
  Serial.begin(9600); // Set the baud rate at 9600 and
                      // Begin serial communication
}
void loop() {

// Read soil moisture display value in monitor

soil_moisture = analogRead(SOIL_MOISTURE_SENSOR );

Serial.write(soil_moisture);

// Start pump when soil moisture is below 500

// Otherwise, turn pump off
if(soil_moisture< MOISTURE_LEVEL) //Adjust value as desired
  digitalWrite(PUMP, HIGH); // Turn pump on
else
  digitalWrite(PUMP, LOW); // Turn pump off
}
```

Discussion/Observation:

Figure 16.1 Soil Moisture Pinout

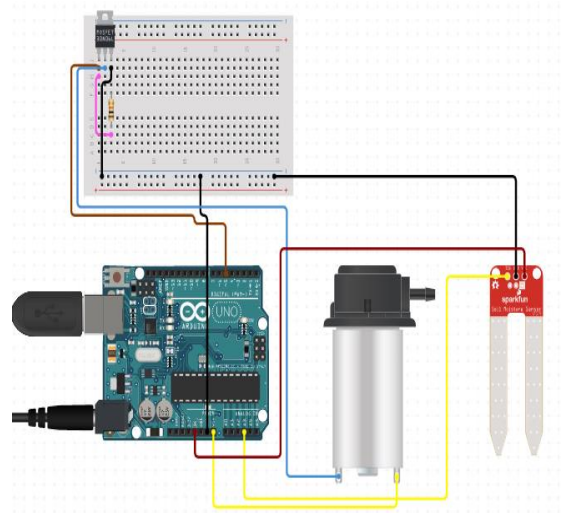


Figure 16.2 Automatic Irrigation System Pictorial Circuit Diagram

Conclusion :

assessment : Design a Simple project in application to soil moisture sensor with high Load.

References :

Experiment 17 PIR Based Burglar Alarm System

Learning Outcome

- 1. Students learn to interface PIR sensor for burglar system.**

Learn to interface PIR sensors in Arduino

Working Principle

Passive infrared (PIR) sensors use a pair of pyroelectric sensors to detect heat energy in the surrounding environment. These two sensors sit beside each other, and when the signal differential between the two sensors changes (if a person enters the room, for example), the sensor will engage. That may mean it triggers an alarm, notifies authorities, or maybe turns on a floodlight. IR radiation focuses on each of the two pyroelectric sensors using a series of lenses constructed as the sensor's housing. These lenses widen the device's sensing area.

Components List

- 1 Arduino
- 1 PIR sensor
- 1 Buzzer
- 1 Relay or Transistor
- 1 Breadboard

Sketch

```
// Assign connection to Arduino Uno pins
const int BUZZER = 2;
const int SENSOR = 3;
```

```
// Set up variable
int pirSensorValue;
Figure 16.1 PIR Sensor Pinout
```

```
void setup() {
  pinMode(BUZZER, OUTPUT);
  pinMode(SENSOR, INPUT);
```

```
  Serial.begin(9600); // Set the baud rate at 9600 and
```

```
    // Begin serial communication
```

```
  digitalWrite(BUZZER, LOW); // Buzzer is set to off
```

```
}
```

```
void loop() {
  pirSensorValue = digitalRead(SENSOR); // Read sensor state
  Serial.println(pirSensorValue); // Send sensor state to monitor
  if (pirSensorValue == HIGH) // Motion is detected
    digitalWrite(BUZZER, HIGH); // Turn buzzer on
  else // Motion is not detected
    digitalWrite(BUZZER, LOW); // Turn buzzer off
}
```

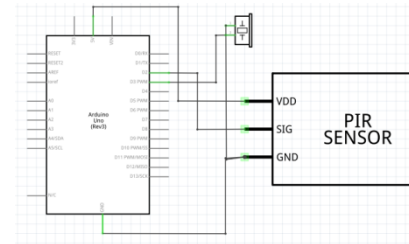


Figure 17.2 PIR Sensor with Buzzer Schematic Diagram

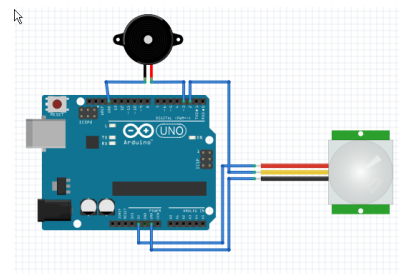


Figure 17.3 PIR Sensor with Buzzer Pictorial Circuit Diagram

Discussion/Observation:

Conclusion :

References :

Experiment 18 Automatic Hand Sanitizer

Components List

- 1 Arduino
- 1 PIR sensor
- 1 Buzzer
- 1 Relay or Transistor
- 1 Breadboard

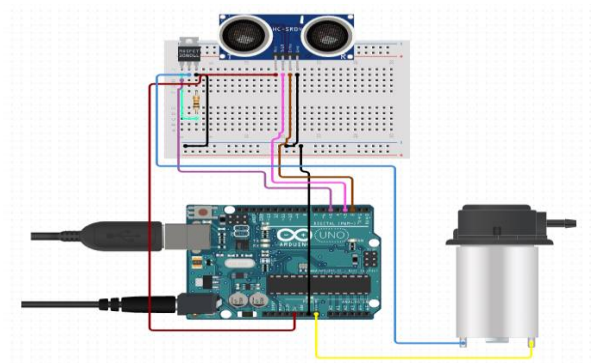


Figure 18.1 : Automatic Hand Sanitizer

Sketch

```
const int TRIGGER = 3;
const int ECHO = 2;
const int SANITIZER_PUMP= 5;

void setup()
{
  pinMode(TRIGGER, OUTPUT); // Set the TRIGGER pin as output
  pinMode(ECHO, INPUT); // Set the ECHO pin as input
  pinMode(SANITIZER_PUMP, OUTPUT); // Set the ZANITIZER_PUMP pin as output
  digitalWrite(SANITIZER_PUMP, LOW); // Set sanitizer pump initially off
}
void loop()
{
  // Sense distance of hand from sensor
  digitalWrite(TRIGGER, LOW); // TRIGGER pin initially set LOW
  delayMicroseconds(2); // for 2 microseconds
  digitalWrite(TRIGGER, HIGH); // Sensor sends out ultrasonic pulses
  delayMicroseconds(5); // for 5 microseconds
  digitalWrite(TRIGGER, LOW); // Sensor stops emitting pulses
  int duration = pulseIn(ECHO, HIGH); // Time in microseconds from when signal
                                     // is sent up to when its echo is received
  int distance = duration / 2 / 29.0 ; // Distance of object from sensor in cm
                                     // is half of round-trip distance traveled by sound
                                     // Sound speed = 29 centimeters/microsecond

  // Dispense sanitizer when hand is within range
  if(distance <= 15) { // Is hand within 15 cm?
    digitalWrite(SANITIZER_PUMP, HIGH); // Turn pump on
    delay(400); // for 400 microseconds
    digitalWrite(SANITIZER_PUMP, LOW); // and turn pump off
    delay(1000); // Pause for 1 sec to give hand time to move away
                // before sensing again for hand presence
  }
}
}
```

Discussion/Observation:

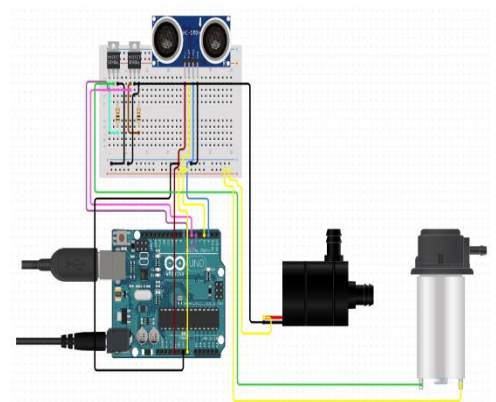
Conclusion :

References :

Experiment 19 Automatic Hand Washer

Components List

- 1 Bread Board
- Jumper Wires
- 1 Relay Module or Transistor
- 1 Ultrasonic
- 1 Arduino



2 Motor Pump (Submersible)

Sketch

```
// Assign connection to Arduino Uno pins
const int TRIGGER = 3;
const int ECHO = 2;
const int WATER_PUMP = 6;
const int SOAP_PUMP = 5;
void setup()
{pinMode(TRIGGER, OUTPUT); // Set the TRIGGER pin as output
pinMode(ECHO, INPUT); // Set the ECHO pin as input
pinMode(WATER_PUMP, OUTPUT); // Set the WATER_PUMP pin as output
pinMode(SOAP_PUMP, OUTPUT); // Set the SOAP_PUMP pin as output
digitalWrite(WATER_PUMP, LOW); // Water pump initially switched off
digitalWrite(SOAP_PUMP, LOW); // Soap pump initially switched off
}

void loop()
{// Sense distance of user from sensor
digitalWrite(TRIGGER, LOW); // TRIGGER pin initially set LOW
delayMicroseconds(2); // for 2 microseconds
digitalWrite(TRIGGER, HIGH); // Sensor sends out ultrasonic pulses
delayMicroseconds(5); // for 5 microseconds
digitalWrite(TRIGGER, LOW); // Sensor stops emitting pulses
int duration = pulseIn(ECHO, HIGH); // Time in microseconds from when signal
// is sent up to when its echo is received
int distance = duration / 2 / 29.0; // Distance of object from sensor in cm
// is half of round-trip distance traveled by sound
// Sound speed = 29 centimeters/microsecond
// Dispense water, soap, then water when user is within specified range
if(distance <= 40){ // Is person within 40 cm?
digitalWrite(WATER_PUMP, HIGH); // Turn water pump on
delay(400); // for 400 microseconds
digitalWrite(WATER_PUMP, LOW); // and turn it off
delay(200); // Pause for 200 microseconds
digitalWrite(SOAP_PUMP, HIGH); // Turn soap pump on
delay(400); // for 400 microseconds
digitalWrite(SOAP_PUMP, LOW); // and turn it off
delay(10000); // Let user rub hands for 10 secs

digitalWrite(WATER_PUMP, HIGH); // Turn water pump on
delay(18000); // for 18 seconds
digitalWrite(WATER_PUMP, LOW); // and turn it off
delay(2000); // Pause for 2 seconds to let use move away
// before sensing for next user
}
}
```

Figure 19.1 : Single Station Automatic Hand Wash

Discussion/Observation:

Conclusion :

[illegible]

References :

Experiment 20

Protective System for the blind

Learning Outcomes

1. Students learn to interface and program Know how ultrasonic senso use as a protective system .

This Activity uses two sensor that can be Attached to blind person Stick , Shoes , Body or Cap to alert the person of obstacles

Components list

- 1 Arduino
- 1 Buzzer
- 2 Ultrasonic
- 1 Relay Module or Transistor
- 1 Bread Board
- Jumper Wires

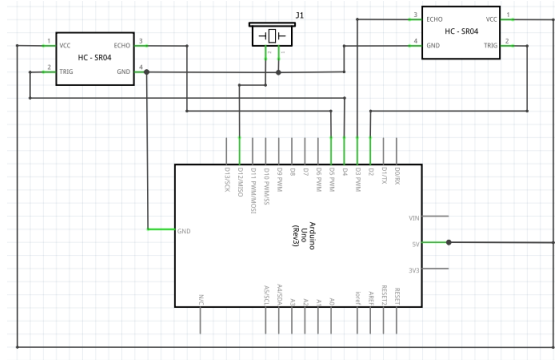


Figure 20.1 : Stick for the Blind

Schematic Diagram

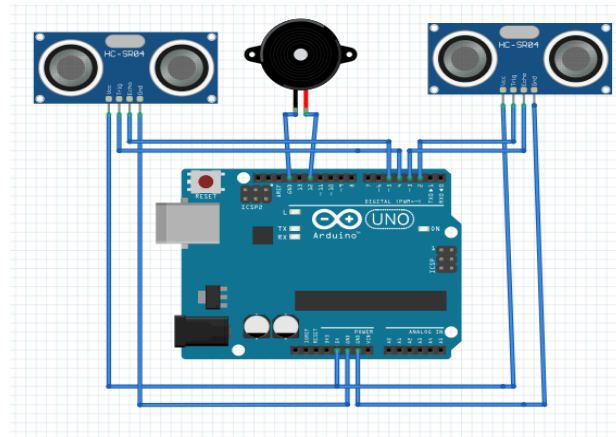


Figure 20.2 : Stick for the Blind Pictorial

Circuit Diagram

Sketch

```
// Assign connection to Arduino Uno pins
const int BUZZER = 2;
const int TRIGGER1 = 4;
const int ECHO1 = 3;
const int TRIGGER2 = 6;
const int ECHO2 = 5;
const int MAX_OBSTACLE_DISTANCE = 60;
//Change value to suit need
// Set up variables
int duration;
int distance;
void setup() {
  // Set the digital pins as output/input
  pinMode(TRIGGER1, OUTPUT);
  pinMode(ECHO1, INPUT);
  pinMode(TRIGGER2, OUTPUT);
```

```

pinMode(ECHO2, INPUT);
pinMode(BUZZER, OUTPUT);
Serial.begin(9600); // Set the baud rate at 9600 and
                    // Begin serial communication
}
// Detects obstacle and sounds alarm
// if obstacle is within MAX_OBSTACLE_DISTANCE
void processSensorData(int trigger_pin, int echo_pin) {
// Sense distance of obstacle from sensor
digitalWrite(trigger_pin, LOW); // TRIGGER pin initially set LOW
delayMicroseconds(2);           // for 2 microseconds
digitalWrite(trigger_pin, HIGH); // Sensor sends out ultrasonic pulses
delayMicroseconds(5);           // for 5 microseconds
digitalWrite(trigger_pin, LOW); // Sensor stops emitting pulses
duration = pulseIn(echo_pin, HIGH); // Time in microseconds from when signal
                                     // is sent up to when its echo is received
distance = duration / 2 / 29.0; // Distance of object from sensor in cm
                                // is half of round-trip distance traveled by sound
                                // Sound speed = 29 centimeters/microsecond

// Sound alarm if obstacle is within MAX_OBSTACLE_DISTANCE
if (distance < MAX_OBSTACLE_DISTANCE) {
    digitalWrite (BUZZER, HIGH); //Turn buzzer on
    delay(200);                  // for 200 ms
    digitalWrite (BUZZER, LOW);  // and turn it off
}
}

void loop() {
// Activate sensor 1 and process data
processSensorData(TRIGGER1, ECHO1);

// Activate sensor 2 and process data
processSensorData(TRIGGER2, ECHO2);

delay(10); // Wait for 10 ms before sensing
           // and processing data again
}

```

Discussion/Observation:

Conclusion :

References :

1. Student learn to program sim 900 model to interface in Arduino.

Components List

- 1 Arduino
- 1 GMS Module Sim900A
- 1 Ultrasonic
- 1 Relay Module or Transistor
- 1 Bread Board
- Jumper Wires

Circuit Diagram

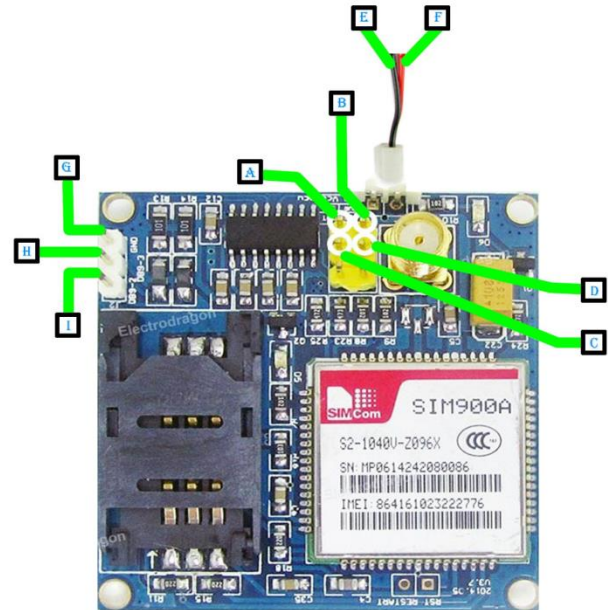
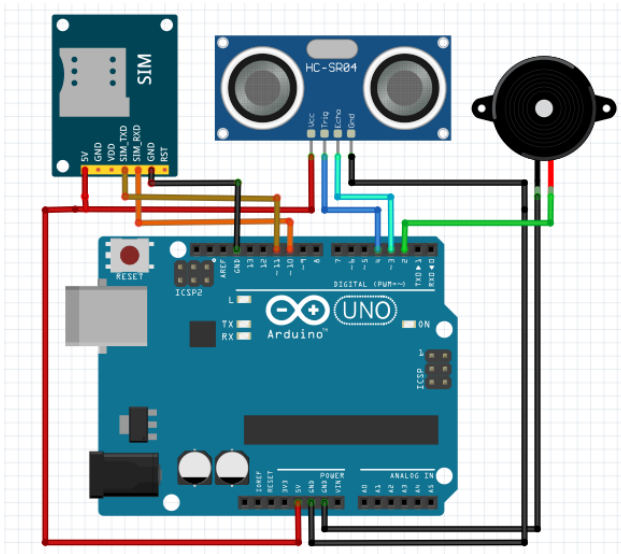
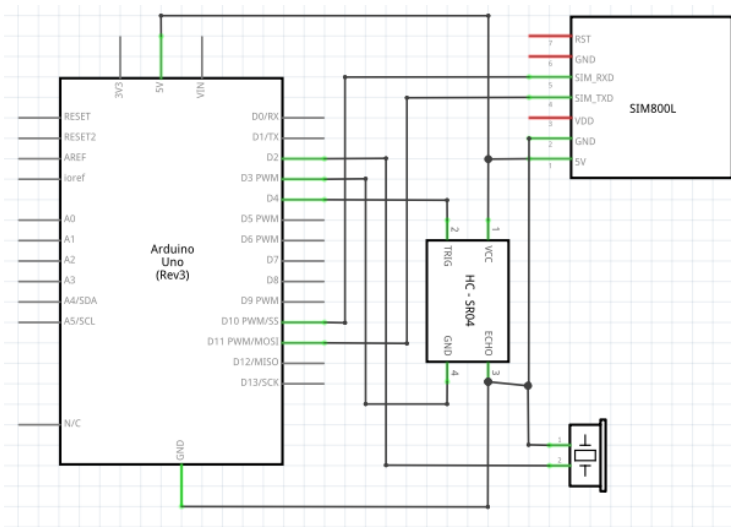


Figure 21.1: Water Level Alerting system – SMS

Sim 900 A PIN OUT

- A.VCC-MCU
- B. GND
- C. 5 VT
- D. 5 VR
- E. VCC
- F. Ground
- G. Gound
- H. RS 232 RXD
- I. RS 232 TXD



Sketch

```
// Include libraries used for LCD and Software Serial communication
#include <SoftwareSerial.h>
#include<LiquidCrystal.h>

// Create SoftwareSerial object, specifying pins 10 and 11 as port
SoftwareSerial mySerial(10,11); // Digital pin 10 used as RX pin
                                // Digital pin 11 used as TX pin

// Assign connection to Arduino Uno pins
const int BUZZER = 2;
const int TRIGGER = 4;
const int ECHO = 3;

// Set up constants, values in cm
const int FULL_DEPTH = 100; // Depth in cm
const int MIN_DEPTH = 20; // Depth in cm
const int SENSOR_CLEARANCE = 30; //Distance above water
                                // surface when tank full

// Set up variables
int distance; // water surface distance from sensor
int duration;

void setup() {
{
Serial.begin(9600); // Set the baud rate at 9600 and
                  // Begin serial communication
mySerial.begin(9600); // Set the baud rate at 9600 and begin
                    // software serial communication
}

void loop() {
{
checkWaterLevel();
}
void checkWaterLevel()
{
// Sense water depth decrease
digitalWrite(TRIGGER, LOW); // TRIGGER pin initially set LOW
delayMicroseconds(2);      // for 2 microseconds
digitalWrite( TRIGGER, HIGH); // Sensor sends out ultrasonic pulses
delayMicroseconds(5);      // for 5 microseconds
digitalWrite(TRIGGER, LOW); // Sensor stops emitting pulses
duration = pulseIn(ECHO, HIGH); // Time in microseconds from when signal
                                // is sent up to when its echo is received
distance = duration / 2 / 29.0 ; // Distance of surface from sensor in cm
                                // is half of round-trip distance traveled by sound
                                // Sound speed = 29 centimeters/microsecond

// If water level is at or below MIN_DEPTH
if(distance <= FULL_DEPTH + SENSOR_CLEARANCE - MIN_DEPTH);
```

```

digitalWrite(BUZZER, HIGH); // Sound alarm
SendTextMessage(); // Send Text message
delay(30000); // Wait for 30 seconds before sending another
                // message, if tank level is not raised above minimum
}
}
void SendTextMessage() {
{
mySerial.println("AT+CMGF=1");
delay(2000); //Required to wait for proper signal transmission .
mySerial.println("AT+CMGS=\"+xxxxxx\""); // Replace number xxxxxx with your phone's
delay(2000);
mySerial.println("Check the Water Level !"); // Send SMS message
                // Change message content as desired

delay(200);
mySerial.println((char)26); // Send message stopping character
delay(5000);
}
}

```

Discussion/Observation:

Conclusion :

Assessment :

References :

Non-Contact Temperature Reader

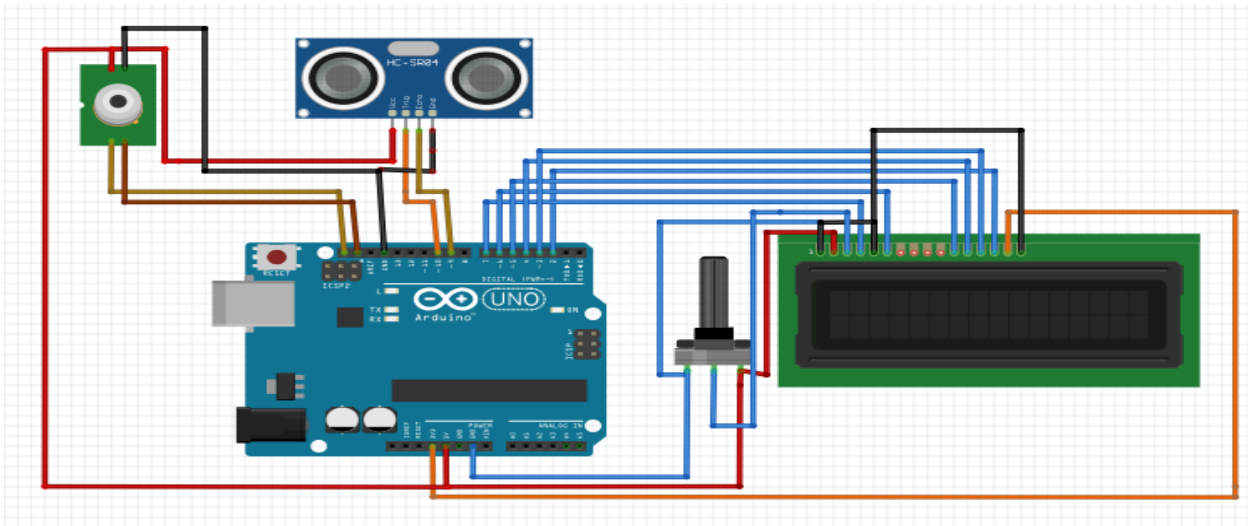


Figure 22.2 : Infrared Temperature sensor Pictorial Circuit Diagram

```
// Include libraries used for LCD and Temperature Sensor
#include <Wire.h>
#include <Adafruit_MLX90614.h>
#include <LiquidCrystal.h>

// Create LiquidCrystal and Adafruit_MLX90614 objects and assign connection to Arduino Uno pins
LiquidCrystal lcd(7,6,5,4,3,2);
Adafruit_MLX90614 mlx = Adafruit_MLX90614();

// Assign connection to Arduino Uno pins
const int TRIGGER = 10;
const int ECHO = 9;

// Set up variables
long duration;
int distance;

// Setup constants
const int MIN_DIST; // Distance above which temperature is read
const int MAX_DIST; // Distance below which temperature is read

void setup() {
  Serial.begin(9600); // Begin serial communication
                    // Set the baud rate at 9600
  // Initialize the LCD screen interface and
  // Specify the width and height of the display
  lcd.begin(16, 2);

  // Initialize the Adafruit_MLX90614 interface
  mlx.begin();

  pinMode(TRIGGER, OUTPUT); // Set the TRIGGER pin as output
  pinMode(ECHO, INPUT);    // Set the ECHO pin as input
}

void loop() {
  // Sense distance of person from sensor
  digitalWrite(trigger_pin, LOW); // TRIGGER pin initially set LOW
```

```

delayMicroseconds(2);           // for 2 microseconds
digitalWrite( trigger_pin HIGH); // Sensor sends out ultrasonic pulses
delayMicroseconds(5);           // for 5 microseconds
digitalWrite(trigger_pin, LOW);  // Sensor stops emitting pulses
duration = pulseIn(echo_pin, HIGH); // Time in microseconds from when signal
                                     // is sent up to when its echo is received
distance = duration / 2 / 29.0 ; // Distance of person from sensor in cm
                                     // is half of round-trip distance traveled by sound
                                     // Sound speed = 29 centimeters/microsecond

// Read and display temperature if person is within range
if ((distance >= MIN_DIST) && (distance <= MAX_DIST)) {
  lcd.clear(); // Clears LCD screen
  lcd.print("Temperature:"); // Display Temperature on first line
  lcd.setCursor(0, 1);       // Set cursor on first column, second line
  lcd.print(mlx.readObjectTempC()); // and display temperature value
  lcd.println("°C");          // followed by °C
  delay(1000); // Put display on for at least 1 second
}
}

```

Discussion/Observation:

Conclusion :

References :

Experiment 23

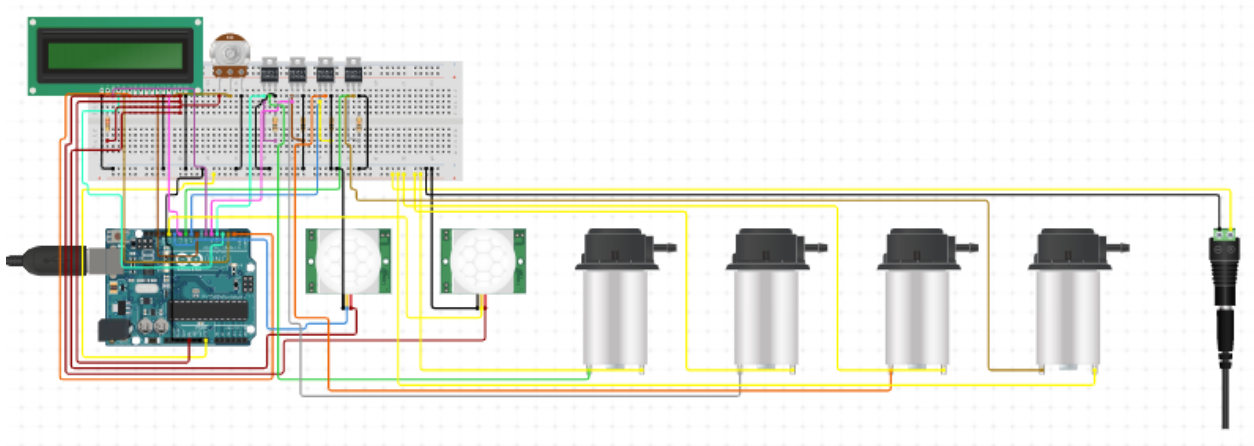
Automated Disinfection Cabin

Learning Outcomes

1. Students learn how to interface multi-input and multi-output
2. Students learn how to program multi-input and multi-output
3. Students learn to adjust the parameters based on the desired output.

Components List

1 Arduino
2 Motion Sensor
1 4 Channel Relay Module
4xMotor Pump
Bread Board



Sketch

```
#define LCD_PIN_RS    11
#define LCD_PIN_E     8
#define LCD_PIN_DB4   2
#define LCD_PIN_DB5   3
#define LCD_PIN_DB6   4
#define LCD_PIN_DB7   7
#include <LiquidCrystal.h>
LiquidCrystal mylcd(LCD_PIN_RS,LCD_PIN_E,LCD_PIN_DB4,LCD_PIN_DB5,LCD_PIN_DB6,LCD_PIN_DB7);
int PERISTALTICPUMP1 = 5;
int PERISTALTICPUMP2 = 6;
int PERISTALTICPUMP3 = 9;
int PERISTALTICPUMP4 = 10;
int PIR_1 = 12;
int PIR_2 = 13;
int pirState = LOW;      // at start, assuming no motion detected
int pirStatus1 = 0;      // variable for reading the status of sensor 1
```

```

int pirStatus2= 0;          // variable for reading the  status of sensor 2
void setup() {
  pinMode(PERISTALTICPUMP1, OUTPUT);  // declare Relay as output
  pinMode(PERISTALTICPUMP2, OUTPUT);  // declare Relay as output
  pinMode(PERISTALTICPUMP3, OUTPUT);  // declare Relay as output
  pinMode(PERISTALTICPUMP4, OUTPUT);  // declare Relay as output
  pinMode(PIR_1, INPUT);  // declare sensor as input
  pinMode(PIR_2, INPUT);
  mylcd.begin(16, 2);
}
void loop(){
  pirStatus1= digitalRead(PIR_1); // read input value
  pirStatus2 = digitalRead(PIR_2); // read input value
  if (pirStatus1== HIGH) {          // check if the input is HIGH
    mylcd.print("Welcome ");
    digitalWrite(PERISTALTICPUMP1, HIGH); // turn Relay ON
    digitalWrite(PERISTALTICPUMP2, HIGH); // turn Relay ON
    digitalWrite(PERISTALTICPUMP3, HIGH); // turn Relay ON
    digitalWrite(PERISTALTICPUMP4, HIGH); // turn Relay ON
  }
  if (pirStatus2== HIGH) {
    digitalWrite(PERISTALTICPUMP1, LOW); // turn Relay OFF
    digitalWrite(PERISTALTICPUMP2, LOW); // turn Relay OFF
    digitalWrite(PERISTALTICPUMP3, LOW); // turn Relay OFF
    digitalWrite(PERISTALTICPUMP4, LOW); // turn Relay OFF
  }
}

```

Discussion/Observation:

Conclusion :

References :

Experiment 24

GSM-Based Security System

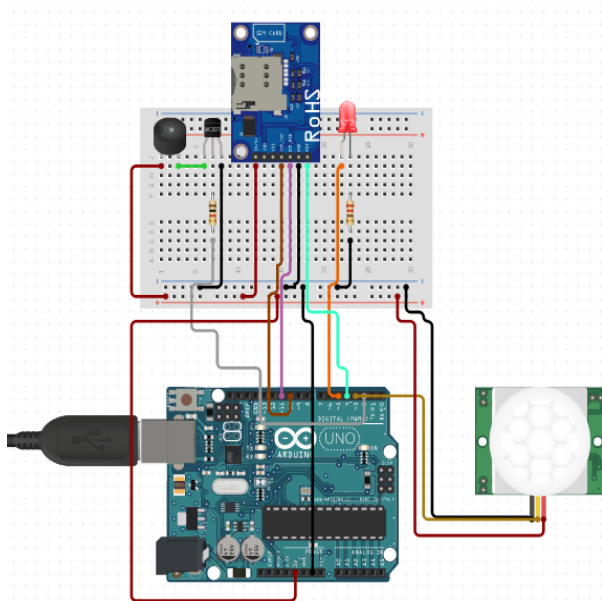
1. Learn how to use GSM/GPRS shield
2. Learn how to use PIR sensor
3. Learn to interface multiple input and multiple output.

Components List

1 Arduino
1 GSM Module Sim900A
1 PIR Sensor
1 Red LED
1 Buzzer
1 Relay Module or Transistor
1 Bread Board
Jumper Wires

Circuit Diagram

Figure 24.1: GSM Based Security System



Sketch

```
// Include library used for Software Serial communication
#include <SoftwareSerial.h>
// Create SoftwareSerial object mySerial, specifying Arduino
pins 10 and 11 as port
SoftwareSerial mySerial(10,11); // Digital pin 10 used as RX
pin                               // Digital pin 11 used as TX

pin
// Assign connection to Arduino Uno pins
const int BUZZER = 2;
const int SENSOR = 3;
const int LED = 5;
// Set up variable
int pirSensorValue
void setup() {
    Serial.begin(9600); // Begin serial communication
                        // Set the baud rate at 9600
    pinMode(BUZZER, OUTPUT); // Set the BUZZER pin as
output
    pinMode(LED, OUTPUT); // Set the LED pin as output
    pinMode(SENSOR, INPUT); // Set the SENSOR pin as input
}
void loop() {
    pirSensorValue = digitalRead(SENSOR); // Read sensor state
    Serial.println(pirSensorValue); // Send sensor state to
monitor
    if (pirSensorValue == HIGH){ // Motion is detected
        SendTextMessage(); // Function to send AT Comds to
GSM mole
        digitalWrite(BUZZER, HIGH); // Turn Sound alarm on
        digitalWrite(LED, HIGH); // Turn visual alarm on
    }
    else {
        digitalWrite(BUZZER, LOW);
        digitalWrite(LED, LOW);
    }
}

void SendTextMessage() {
    mySerial.println("AT+CMGF=1");
    delay(2000);
    mySerial.println("AT+CMGS=\"+xxxxxx\"\\r"); // Replace
number xxxxxx with your phone's
    delay(2000);
    mySerial.println("Please check for intruder!"); // Send SMS
message
                                // Change message content as desired
    delay(200);
    mySerial.println((char)26); // Send message stopping
character
    delay(5000);
}
```


Discussion/Observation:

Conclusion :

References :

Experiment 25

COVID 19 Patient Tracking System

Learning Outcomes

1. Learn how to interface with Arduino mega
2. Learn how to use GSM/GPRS shield
3. Learn how to use GPS
4. Learn to interface multiple input and multiple output.

List of Components

- 1 Arduino MEGA
- 1 GSM/GPRS Shield (Sim900A)
- 1 GPS (I used the NEO 6, very popular in ebay)
- 1 9V Battery
- 1 Plastic Box
- Some wires

Circuit Diagram

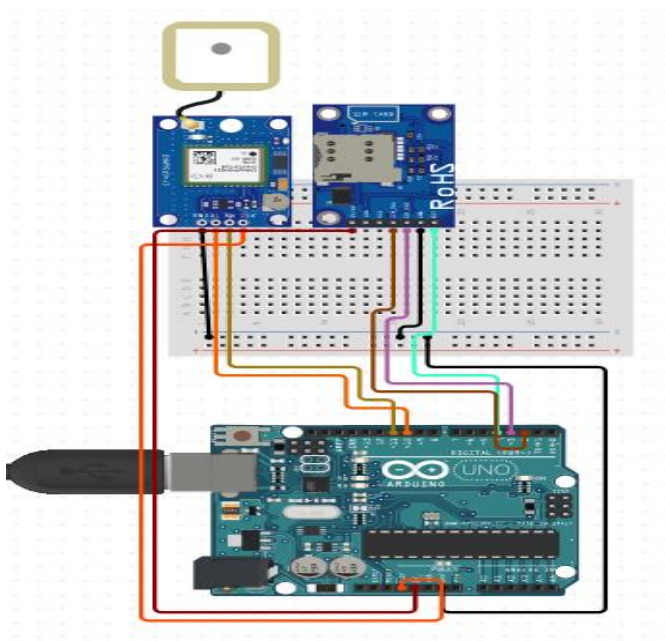


Figure 25.1: COVID 19 Patient Tracking System

```
#include <TinyGPS++.h>
#include <SoftwareSerial.h>
static const int ReceiverPin = 11, TransmitterPin =10;
static const uint32_t GPSBaud = 9600;
// The TinyGPS++ object
TinyGPSPlus gps;
int temp=0,i;
// The serial connection to the GPS device
SoftwareSerial gpsTracker(ReceiverPin,TransmitterPin);
String stringVal = "";
void setup(){
  Serial.begin(9600);
```

```

gpsTracker.begin(GPSBaud);
pinMode(13,OUTPUT);
digitalWrite(13,LOW);
gsm_check();
Serial.println("AT+CNMI=2,2,0,0,0");
temp=0;
}
void loop()
{ serialEvent();
  while(temp)
  { while (gpsTracker.available() > 0)
    { gps.encode(gpsTracker.read());
      if (gps.location.isUpdated())
      { temp=0;
        digitalWrite(13,HIGH);
        trackingPatient();
      } if(!temp)
        break;
    } }
  digitalWrite(13,LOW);
}void serialEvent()
{ while(Serial.available()>0)
  { if(Serial.find("Track Patient"))
    { temp=1;
      break;
    } else
    { temp=0;
    } }}void gsm_check()
{ boolean at_flag=1;
while(at_flag)
{
  Serial.println("AT");
  delay(1);
  while(Serial.available()>0)
  {
    if(Serial.find("OK"))
      at_flag=0;
  }

  delay(1000);
}
boolean echo_flag=1;
while(echo_flag)
{
  Serial.println("ATE0");
  while(Serial.available()>0)
  {
    if(Serial.find("OK"))
      echo_flag=0;
  }
  delay(1000);
}
boolean net_flag=1;
while(net_flag)
{
  Serial.println("AT+CPIN?");
  while(Serial.available()>0)
  {
    if(Serial.find("+CPIN: READY"))
      net_flag=0;
  }
  delay(1000);
}
void start_sms()
{
  Serial.println("AT+CMGF=1");
  delay(400);
  Serial.println("AT+CMGS=\"+251xxxxxxxxx\"");
  // use your 10 digit cell no. here
  delay(400);
}
void send_data(String message)
{
  Serial.print(message);
  delay(200);
}
void send_sms()
{
  Serial.write(26);
}
void trackingPatient()
{
  start_sms();
  send_data("Patient Tracking System:");
  Serial.println(" ");
  send_data("Current Location is:");
  Serial.println(" ");
  Serial.print("Latitude: ");
  Serial.print(gps.location.lat(), 6);
  Serial.print("\n Longitude: ");
  Serial.println(gps.location.lng(), 6);

  //
  https://www.google.com/maps/@8.2630696,77.3022
  699,14z
  Serial.print("https://www.google.com/maps/@");
  Serial.print(gps.location.lat(), 6);
  Serial.print(',');
  Serial.print(gps.location.lng(), 6);
  Serial.print(",14z");
  send_sms();
  delay(2000);
}

```

Discussion/Observation:

Conclusion :

References :

Experiment 26

GSM based Automatic Plant Irrigation System

Learning Outcomes

1. Learn how to interface with high load
2. Learn how to use Quadband GSM/GPRS shield
3. Understand N-Channel MOSFET
4. Learn to interface multiple input and multiple output.

Components List

Arduino Uno
 Submersible Pool Water Pump 240L/H
 LCD 16x2
 220 Ohm Resistor
 Rotary Potentiometer - 10k Ohm, Linear
 Wall Adapter Power Supply - 12VDC 2A
 N-Channel MOSFET 60V 30A
 10K Ohm Resistor
 QuadBand GPRS-GSM SIM900A
 USB Cable A to B
 BreadBoard

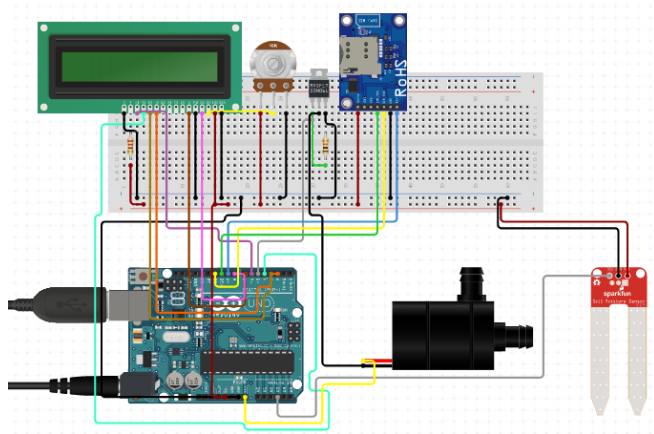


Figure 26.1: GSM based Automatic Plant Irrigation System

Sketch

```

// Include libraries used for LCD and Software Serial
communication
#include<SoftwareSerial.h>
#include<LiquidCrystal.h>

// Create SoftwareSerial object mySerial, specifying Arduino
pins 10 and 11 as port
SoftwareSerial mySerial(10,11); // Digital pin 10 used as
RX pin
                                // Digital pin 11 used as
TX pin

// Create LiquidCrystal object mylcd and
// Assign connection to Arduino Uno pins
LiquidCrystal lcd(8,7,2,3,4,6);

// Assign connection to Arduino Uno pins
const int MOTOR= 5;
const int SENSOR=A3;

// Set up variables
int initial = 0;
  
```

```

String str = "";
void setup() {
  lcd.begin(16, 2); // Initialize the LCD screen interface and
                    // Specify the width and height of the
display
  Serial.begin(9600); // Begin serial communication
                    // Set the baud rate at 9600
  pinMode(MOTOR, OUTPUT); // Set the MOTOR pin as
output
  pinMode(SENSOR, INPUT); // Set the SENSOR pin as Input
  lcd.print("Automatic Irrigaton");

  gsmInit();
}

void loop() {
  {
    if(digitalRead(SENSOR) == 1 && initial == 0) {
      delay(1000);
      if(digitalRead(SENSOR)==1) {
        sendSMS("Motor Will Be ON : Less Moisture ");
        lcd.begin(16,2);
        lcd.setCursor(0,1);
        lcd.print("Motor ON  ");
        digitalWrite(MOTOR, HIGH);
        delay(1000);
        initial=1;
      }
    }
  }
}
  
```

```

else if(digitalRead(SENSOR) == 0 && initial == 1) {
  delay(500);
  if(digitalRead(SENSOR)==0) {
    sendSMS("Motor turned OFF: Soil Moisture is Normal ");
    digitalWrite(MOTOR, LOW);
    lcd.begin(16,2);
    lcd.print("Motor OFF");
    lcd.setCursor(0,1);
    lcd.print("Motor OFF");
    mySerial.println("AT+CMGF=1");
    delay(500);
    mySerial.print("AT+CMGS=");
    mySerial.print("");
    mySerial.print("+251xxxxxxxxx"); // number
    mySerial.print("");
    mySerial.println();
    delay(500);
    mySerial.println(msg);
    delay(500);
    mySerial.write(26);
    delay(1000);
    lcd.clear();
    lcd.print("SMS Sent");
    delay(1000);
    lcd.begin(16, 2);
  }

  void gsmlnit() {
    lcd.clear();
    lcd.print("Finding Module..");
    boolean at_flag = 1;
    while(at_flag) {
      mySerial.println("AT");
      while(mySerial.available() > 0) {
        if(mySerial.find("OK"))
          at_flag = 0;
      }
    }

    delay(1000);
    initial=0;
  }
}

void sendSMS(String msg) {
  lcd.clear();
  lcd.print("Sending SMS");
}
delay(1000);
}
mySerial.println("ATE0");
lcd.clear();
lcd.print("Finding Network..");
boolean net_flag = 1;
while(net_flag) {
  mySerial.println("AT+CPIN?");
  while(mySerial.available() > 0) {
    if(mySerial.find("READY"))
      net_flag=0;
    break;
  }
  delay(1000);
}
mySerial.println("AT+CNMI=2,2,0,0,0");
delay(1000);
mySerial.println("AT+CMGF=1");
delay(1000);
mySerial.println("AT+CSMP=17,167,0,0");
lcd.clear();
mySerial.flush();
}

```

Discussion/Observation:

Conclusion :

References :

Experiment 27

Anti-Drunk Car Driving System

Learning Outcomes

1. Learn how to interface with high load
2. Learn how to use Quadband GSM/GPRS shield
3. Understand N-Channel MOSFET
4. Learn to interface multiple input and multiple output.

Components List

- 1 Arduino UNO
- 1 GSM SIM 900A
- 1 DC Motor
- 1 LED Red
- 1 Alcohol Sensor (MQ-3)
- 1 Speaker
- 1 Breadboard
- Some wires

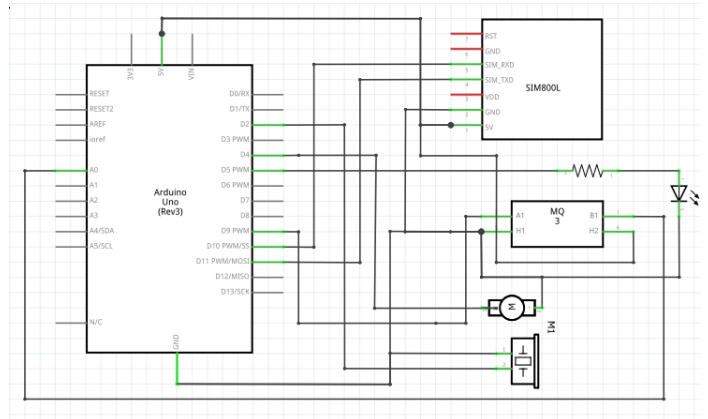


Figure 27.1: Schematic Diagram

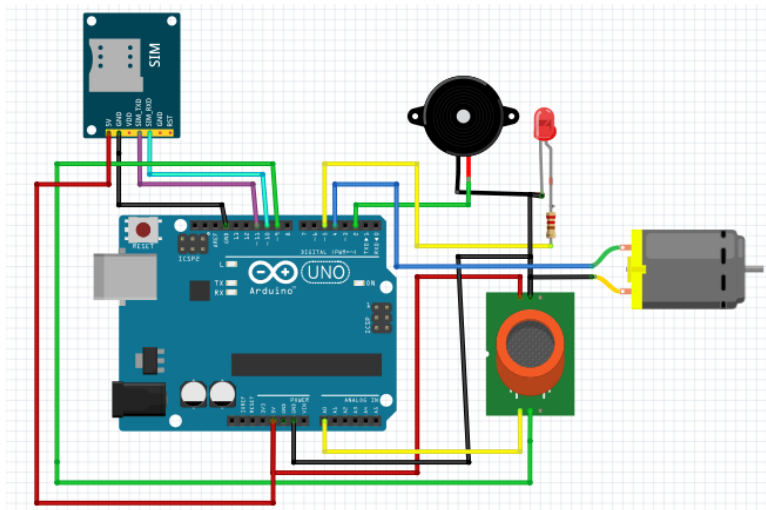


Figure 27.2: Pictorial Circuit Diagram

Sketch

```
// Include library used for Software Serial communication
#include <SoftwareSerial.h>
// Create SoftwareSerial object mySerial, specifying Arduino pins 10 and 11 as port
SoftwareSerial mySerial(10,11); // Digital pin 10 used as RX pin
                                // Digital pin 11 used as TX pin

const int ALCOHOL_LEVEL=10;
const int SENSOR = A0; // Mq3 gas Sensor
const int PILOTLAMP = 2;
const int ENABLE_SWITCH = 4;
```



```

const int BUZZER = 2;
int value;

void setup() {
  Serial.begin(9600); // Begin serial communication
                      // Set the baud rate at 9600
  pinMode(SENSOR, INPUT); // Set the SENSOR pin as input
  pinMode(ENABLE_SWITCH, OUTPUT); // Set the MOTOR pin as output
  pinMode(BUZZER, OUTPUT); // Set the BUZZER pin as output
  pinMode(PILOTLAMP, OUTPUT); // Set the BUZZER pin as output

}
void loop() {
{
  value= analogRead(SENSOR); // reads the analog value from the alcohol sensor's AOUT pin
  Serial.println(value); // prints the alcohol value
  delay(100);
  if (value > ALCOHOL_LEVEL) { // Is alcohol level above allowable ?
    digitalWrite(ENABLE_SWITCH, LOW); // Disable Car Engine starter
    SendTextMessage();//Alert Authorities
    digitalWrite(BUZZER, HIGH); // Turn On Sound Alarm
    digitalWrite(PILOTLAMP, HIGH); // Turn On Visual Alarm
    delay(1000); // Wait for 1 second before checking Alcohol level again
  }
  else {
    digitalWrite(ENABLE_SWITCH, HIGH); // Enable Car Engine starter
    digitalWrite(BUZZER, LOW); // Turn Off Sound Alarm
    digitalWrite(PILOTLAMP, LOW); // Turn Off Visual Alarm

  }
}
}
void SendTextMessage() {
{
  mySerial.println("AT+CMGF=1");
  delay(2000);
  mySerial.println("AT+CMGS=\"+xxxxxx\"\\r"); // Replace number xxxxxx with your phone's
  delay(2000);
  mySerial.println("Check the Alcohol Level!"); // Send SMS message
                                              // Change message content as desired
  delay(200);
  mySerial.println((char)26); // Send message stopping character
  delay(5000);
}
}

```

Discussion/Observation:

Conclusion :

References :

Experiment 28

Heart Rate Monitoring and alert System

Learning Outcomes

1. Learn how to interface Pulse Rate Sensor
2. Learn how to interface GSM/GPRS shield
3. Learn to interface multiple input and multiple output.

Components List

1 Arduino Uno
1 LCD 16x2
1 Pulse Rate Sensor
1 Rotary Potentiometer - 10k Ohm, Linear
1 GSM SIM900A
2 Push Button Switch
1 USB Cable A to B
1 BreadBoard
Some wires

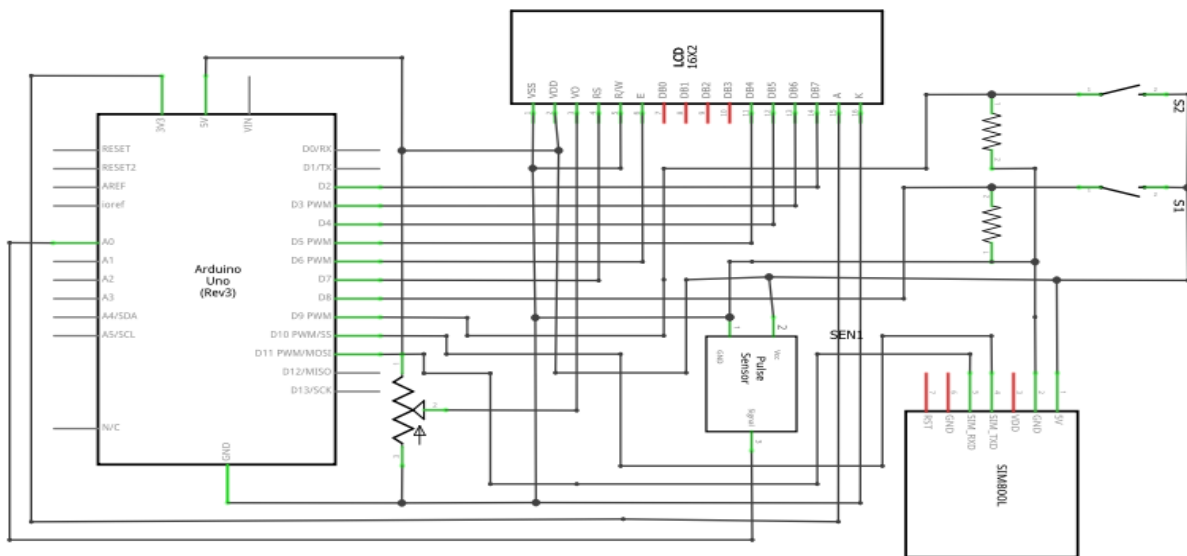
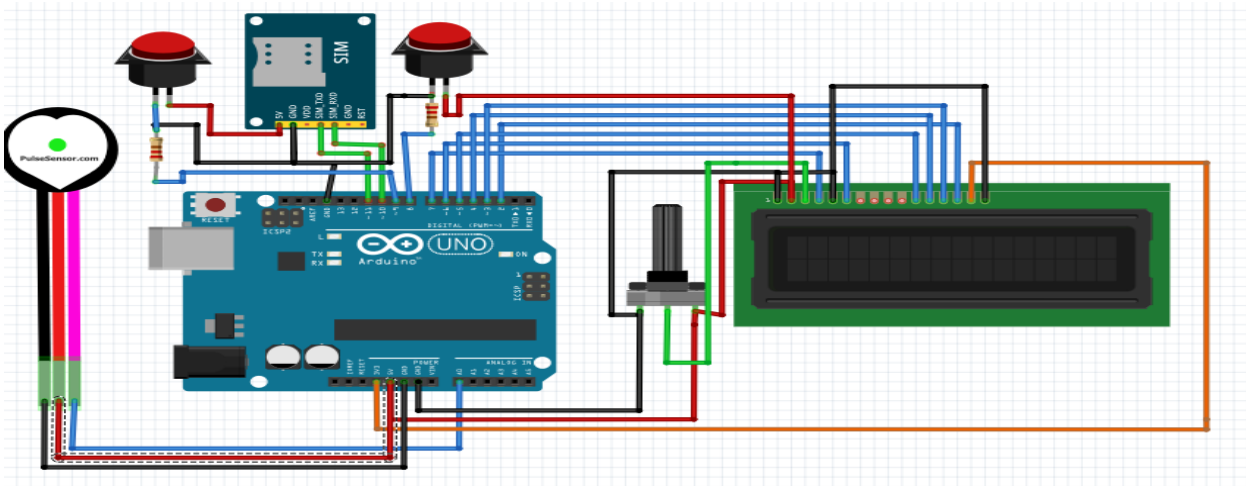


Figure 28.1 : Heart Rate Monitoring System with SMS and Display Schematic Diagram



**Figure 28.2 : Heart Rate Monitoring System
with SMS and Display Pictorial Circuit Diagram**

Sketch

```
#include <SoftwareSerial.h>
SoftwareSerial joSerial(10,11);
// Include libraries used for LCD and Software Serial
communication
#include <SoftwareSerial.h>
//SoftwareSerial object mySerial, specifying Arduino
pins 10 and 11 as port
// Digital pin 10 used as RX pin
// Digital pin 11 used as TX pin
SoftwareSerial mySerial(10,11);
#include<LiquidCrystal.h>
```

```
// Create LiquidCrystal object lcd and
// Assign connection to Arduino Uno pins
LiquidCrystal lcd(7,6,5,4,3,2);
// Assign connection to Arduino Uno pins
int heartSensor =A0;
int Reset=13;
int start=12;
// Set up variables
int count=0,i=0,k=0,rate=0;
unsigned long time2,time1;
unsigned long time;
byte heart[8] =
{
  0b00000,
  0b01010,
  0b11111,
  0b11111,
  0b11111,
  0b01110,
  0b00100,
  0b00000
};
```

```
void setup()
{
  mySerial.begin(9600);
  lcd.createChar(1, heart);
  // Initialize the LCD screen interface and
  // Specify the width and height of the display
  lcd.begin(16,2);

  lcd.print("Heart Beat "); // Create heart shape and
represent by int 1
  lcd.write(1); // Display heart shape
  lcd.setCursor(0,1); // Set cursor on first column,
second line
  lcd.print("Monitoring");
  pinMode(heartSensor, INPUT); // Set the SENSOR pin
as input
  pinMode(Reset, INPUT); // Set the RESET pin as input
  pinMode(start, INPUT); // Set the START pin as input
  digitalWrite(Reset, HIGH); // Enable internal pull-up
resistor on RESET pin
  digitalWrite(start, HIGH); // Enable internal pull-up
resistor on START pin
  delay(1000);
}
void loop()
{
  if(!(digitalRead(start))) // Start button pressed?
  {
    k=0;
    lcd.clear();
    lcd.print("Please wait.....");
    while(k<5)
    {
      if(digitalRead(heartSensor))
```

```

{
  if(k==0)
    time1=millis(); // Save initial time
  k++;
  while(digitalRead(heartSensor));
}
}
time2=millis();
rate=time2-time1;
rate=rate/5;
rate=60000/rate;
lcd.clear();
lcd.print("Heart Beat Rate:");
lcd.setCursor(0,1);
lcd.print(rate);
lcd.print(" ");
lcd.write(1);

if(rate < 60 && rate > 100){
  SendTextMessage();
  // Reset variables to initial values
  k=0;
  rate=0;
}
if(!digitalRead(Reset))

```

```

{
  rate=0;
  // Compute and display heart beat rate
  lcd.clear();
  lcd.print("Heart Beat Rate:");
  lcd.setCursor(0,1);
  lcd.write(1);
  lcd.print(rate);
  k=0;
}
} void SendTextMessage()
{ mySerial.println("AT+CMGF=1");
//To send SMS in Text Mode
  delay(2000);
  mySerial.println("AT+CMGS=\"+2519xxxxxx\"\\r"); //
change to the phone number you using
  delay(2000);
  mySerial.println("Heart beat rate is outside normal
range!");//the content of the message
  delay(200);
  mySerial.println((char)60);//the stopping character
  delay(5000);
}

```

Discussion/Observation:

Conclusion :

References :

Experiment 29

Digital Visitor Counter System

Learning Outcomes

1. Learn how to interface with high load
2. Learn how to use Quadband GSM/GPRS shield
3. Understand N-Channel MOSFET
4. Learn to interface multiple input and multiple output.

Component List

1 Arduino
1 LCD 16x2
2 220 Ohm Resistor
1 Rotary Potentiometer - 10k Ohm, Linear
1 Relay Module 4-Ch
1 USB Cable A to B
1 LED Green
1 LED Red
2 Infrared Sensor
1 BreadBoard
Jumper Wires Pack

Sketch

```
// Include libraries used for LCD
#include<LiquidCrystal.h>
// Create LiquidCrystal object lcd and
// Assign connection to Arduino Uno pins
LiquidCrystal lcd(11,10,4,7,8,9);
// Assign connection to Arduino Uno pins
const int GREEN_LED = 5;
const int RED_LED = 6;
const int IN_SENSOR = 2;
const int OUT_SENSOR = 3;
const int RELAY = 12;
// Set up and initialize variable
int count = 0;
const int CAPACITY = 100;
void updateDisplay() {
{
lcd.clear();
lcd.print("Person(s) In Room: "); // display "Person(s) In Room: " message
lcd.setCursor(0,1); // Set cursor position to first column, second line
lcd.print(count); // and display initial count
}
}
void setup() {
// Set the digital pins as output
pinMode(GREEN_LED, OUTPUT);
pinMode(RED_LED, OUTPUT);
pinMode(RELAY, OUTPUT);
// Set the digital pins as input
pinMode(IN_SENSOR, INPUT);
```

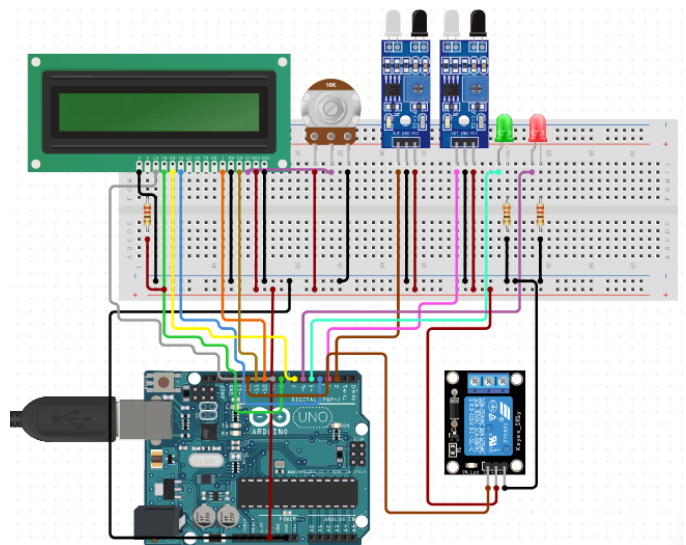


Figure 29.1

Bidirectional Visitor Digital Counter Pictorial Circuit Diagram

```

pinMode(OUT_SENSOR, INPUT);
lcd.begin(16, 2); // Initialize the LCD screen interface and
                // Specify the width and height of the display

lcd.print("Visitor Counter"); // Display "Visitor Counter" message
delay(2000);                  // for 2 seconds
lcd.clear();                  // then clear LCD screen and
lcd.print("Person(s) In Room: "); // display "Person(s) In Room: " message
lcd.setCursor(0, 1); // Set cursor position to first column, second line
lcd.print(count);    // and display initial count
}
void loop() {
{
if(digitalRead(IN_SENSOR)) {
count++;
updateDisplay();
delay(1000); // Wait for a second
}
if(digitalRead(OUT_SENSOR)) {
count--;
updateDisplay();
delay(1000); // Wait for a second
}
if(count <= 0) { // Is room empty?
lcd.clear();
digitalWrite(RELAY, LOW); // Turn off light
lcd.clear();
lcd.print("Nobody In Room");
lcd.setCursor(0, 1);
lcd.print("Light Is Off");
delay(200);
}
else {
digitalWrite(RELAY, HIGH); // Turn on light
if (count <= CAPACITY) { // Is room at or below full capacity?
digitalWrite(GREEN_LED, HIGH); // Turn on green LED
digitalWrite(RED_LED, LOW);    // Turn off red LED
}
else { // Room is above full capacity
digitalWrite(GREEN_LED, LOW); // Turn off green LED
digitalWrite(RED_LED, HIGH);  // Turn on red LED
}
}
}
}

```

Discussion/Observation:

Conclusion :

References :

Experiment 30

Home Automation System

Part 1. RFID Door Lock System with Display

Components List

- 1 Arduino Uno
- 1 Piezo Buzzer
- 1 Transistor - NPN BC337
- 1 1K Ohm Resistor
- 1 12VDC Lock-style Solenoid
- 1 Diode Rectifier - 1A 50V
- 1 RFID - RC522 RF IC Card Sensor Module
- 1 LCD 16x2
- 1 220 Ohm Resistor
- 1 Rotary Potentiometer - 10k Ohm, Linea
- 1 Wall Adapter Power Supply - 12VDC 2A

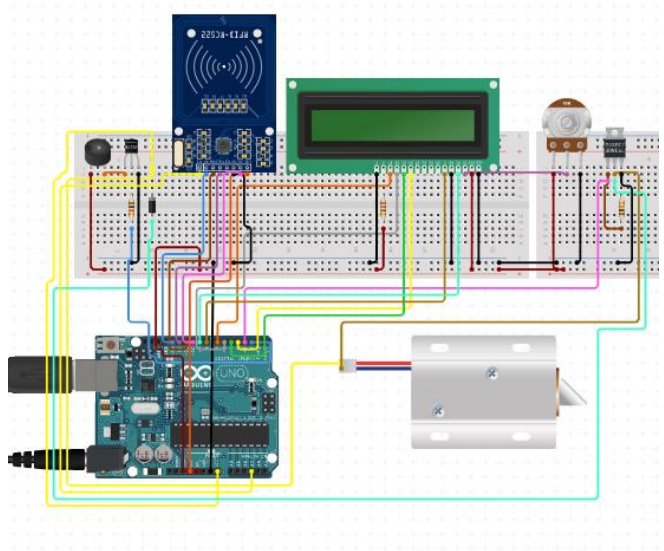


Figure 29.1 RFID Door Lock with Display

- 1 Bread Board
- Some Wires

Sketch

```
#include <SPI.h>
#include <RFID.h>
#define SS_PIN 10
#define RST_PIN A3
LiquidCrystal lcd(9,8, 4, 5, 6, 7);
RFID rfid(SS_PIN,RST_PIN);
int Buzzer =2;
const int LockPin =3;
int serNum[5];
int cards[][5] = {
  {5,117,21,219,190}
};
bool access = false;
void setup(){
  lcd.begin(16, 2); // LCD screen
  Serial.begin(9600);
  SPI.begin();
  rfid.init();
  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);
}
void loop(){
  if(rfid.isCard()){
```

```

if(rfid.readCardSerial()){
  Serial.print(rfid.serNum[0]);
  Serial.print(" ");
  Serial.print(rfid.serNum[1]);
  Serial.print(" ");
  Serial.print(rfid.serNum[2]);
  Serial.print(" ");
  Serial.print(rfid.serNum[3]);
  Serial.print(" ");
  Serial.print(rfid.serNum[4]);
  Serial.println("");

  for(int x = 0; x < sizeof(cards); x++){
    for(int i = 0; i < sizeof(rfid.serNum); i++){
      if(rfid.serNum[i] != cards[x][i]) {
        access = false;
        break;
      } else {
        access = true;
      }
    }
    if(access) break;
  }
}

if(access){
  lcd.print("Welcome!");
  digitalWrite(Buzzer,LOW);
  delay(1000);
  digitalWrite(LockPin,HIGH);

} else {
  Serial.println("Not allowed!");
  digitalWrite(LockPin,HIGH);
  delay(1000);
  digitalWrite(Buzzer,HIGH);
}

rfid.halt();
}

```

Discussion/Observation:

Conclusion :

References :

Mastering Arduino basics in 30 Days by Engr. Jolan B. Sy

Part 2

IR Remote Based Home Automation System

Learning Outcomes

1. Understand the principle of IR Receiver Diode
2. Interface IR Receiver diode in microcontroller

Components List

1 Arduino
1 IR Receiver Diode - TSOP38238
1 LCD 16x2
1 220 Ohm Resistor
1 Rotary Potentiometer - 10k Ohm, Linear
4 Relay Module 4-Ch
1 10 Watts Lamp
1 USB Cable A to B
1 BreadBoard
Jumper Wires Pack

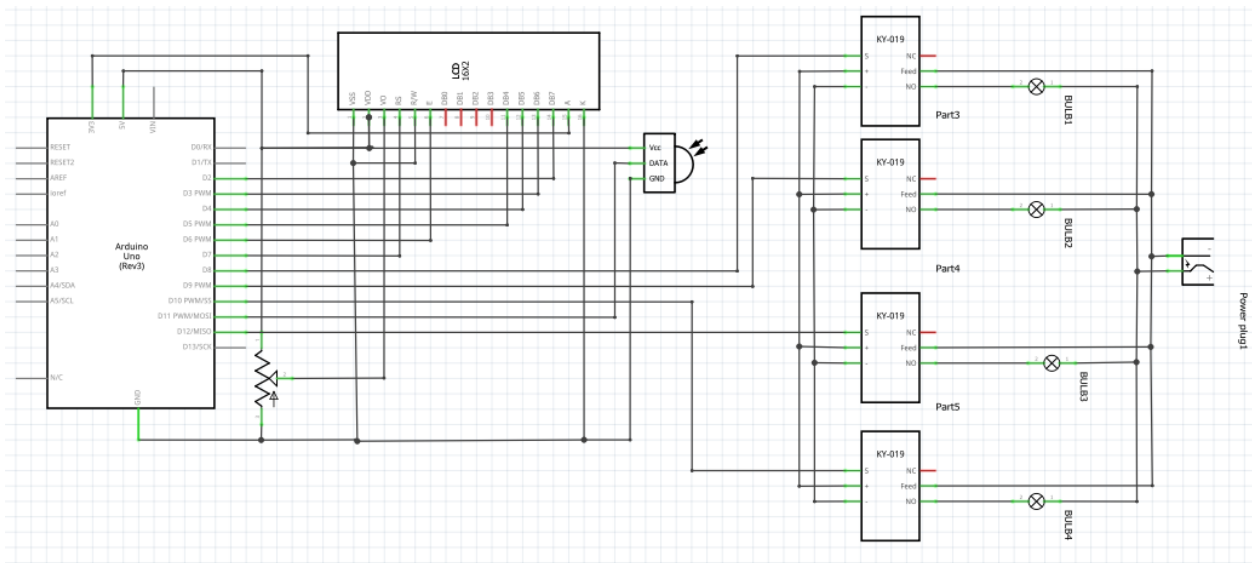


Figure 29.1 IR Remote Based Home Automation System Schematic Diagram

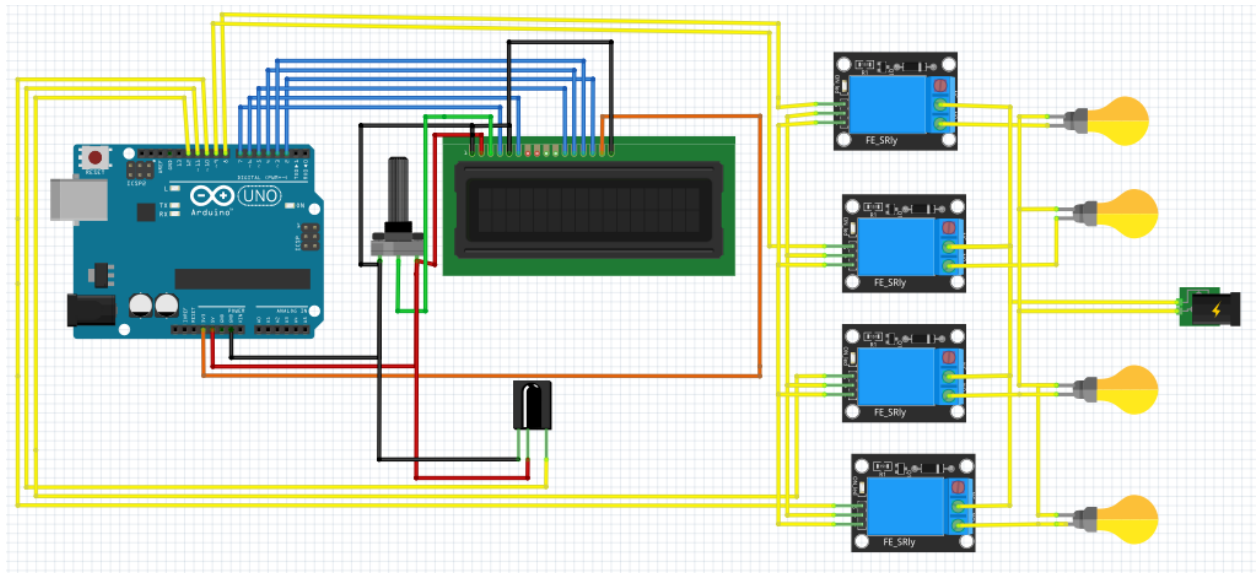


Figure 29.2 IR Remote Based Home Automation System Pictorial Circuit Diagram

Sketch

```
#include<LiquidCrystal.h>
#include <IRremote.h>
const int RECV_PIN=11;
IRrecv irrecv(RECV_PIN);
decode_results results;
#include<LiquidCrystal.h>
LiquidCrystal lcd(7,6,2,3,4,5);

#define Fan 8
#define Light 9
#define TV 10

int i=0,j=0,k=0,n=0;

void setup()
{
  Serial.begin(9600);
  lcd.begin(16,2);
  pinMode(Fan, OUTPUT);
  pinMode(Light, OUTPUT);
  pinMode(TV, OUTPUT);
  //digitalWrite(13,HIGH);
  lcd.print("Home Automation");
  irrecv.enableIRIn(); // Start the receiver
  irrecv.blink13(true);
  lcd.print("Fan Light TV ");
  lcd.setCursor(0,1);
  lcd.print("OFF OFF OFF");
}
```

```
void loop()
{
  if (irrecv.decode(&results))
  {
    Serial.println(results.value,HEX);
    delay(100);
    lcd.setCursor(0,0);
    lcd.print("Fan Light TV");
    if(results.value==0x1FE00FF)
    {
      i++;
      int x=i%2;
      digitalWrite(Fan, x);
      lcd.setCursor(0,1);
      if(x)
        lcd.print("ON ");
      else
        lcd.print("OFF ");
      // delay(200);
    }

    else if(results.value==0x1FEF00F) // key 1
    {
      j++;
      int x=j%2;
      digitalWrite(Light, x);
      lcd.setCursor(6,1);
      if(x)
        lcd.print("ON ");
      else
```

```

    lcd.print("OFF ");
    // delay(200);
}

if(results.value==0x1FE9867)
{
    k++;
    int x=k%2;
    digitalWrite(TV, x);
    lcd.setCursor(13,1);
    if(x)
        lcd.print("ON ");
    else
        lcd.print("OFF");
    // delay(200);
}

    n++;
    int x=n%2;
    digitalWrite(TV, x);
    digitalWrite(Fan,x);
    digitalWrite(Light,x);
    lcd.setCursor(0,1);
    if(x)
        lcd.print("ON  ON  ON ");
    else
        lcd.print("OFF  OFF  OFF");
    //delay(200);
}
    irrecv.resume(); // Receive the next value
    //delay(100);
}

if(results.value==0x1FE48B7)
{

```

Discussion/Observation:

Conclusion :

References :

Mastering Arduino basics in 30 Days by Engr. Jolan B. Sy

ENGR. JOLAN BACCAY SY R.ECE , ME.ECE



Engr. Jolan Baccay Sy has 20 years of work experience in the academe and the industry and a registered Electronics and Communication Engineer in the Philippines. At present he is an Electronics and Communication Assistant Professor III in College of Computing Studies and Information Technology in Isabela State University , Cagayan Campus , He worked as Assistant Professor in the Electrical and Computer Engineering Department of Kombolcha Institute of Technology, Wollo University, Ethiopia. He teaches Data Communication ,Advance Computer Networks , Microwave Engineering and Microcontroller system. He took his Bachelor of Science in Electronics and Communication Engineering (Batch 2001) and Master of Engineering in Electronics and Communication Engineering(Batch 2005) from University of Saint Louis, Tuguegarao City and completed the required academic subjects for Doctor of Philosophy in Development Management Engineering and currently enrolled in University of Saint Louis taking up Doctor of Engineering and Management . He was Electronics and Telecommunication lecturer in University of Saint Louis from 2002-2004, Head of Department of Electro-Technology in Federal TVET Teachers Training Institute (Ethio-China College) under Ministry of Education, Federal Democratic Republic of Ethiopia,He worked as Assistant Professor in Teesside University Middlesbrough ,Tees Valley UK(Extension Program in Bahrain Training Institute) ,Trainor and Quality Internal Verifier of the Electrical and Electronics Department Bahrain Training Institute(BTEC-TVET). He was employed as Trainor and Marketing director in Engineering Reviewers of ASIA. He was Academic Deans in AMA University Laoag Campus –Philippines at the age of 25. His current involvements are in Computer Networking , Programming , Designing Microprocessor Based system and designing wireless sensor network. . Recently he published two books on analog electronics and microcontroller system and published multi-disciple researches.